A Heterogeneous Federated Learning Approach for Decentralized Pest Identification under Non-IID Distributions

by

Md. Rafiul Islam
22201842
Wasi Farabi
22201358
Sadat Shahriar Arko
22201992
Shayonton Hasan
22201763

A thesis submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science and Engineering

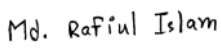
Department of Computer Science and Engineering
Brac University
June 2026.

Declaration

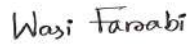
It is hereby declared that

1. The thesis submitted is our own original work while completing degree at Brac University.
2. The thesis does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
3. The thesis does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.
4. We have acknowledged all main sources of help.

Student's Full Name & Signature:



Md. Rafiul Islam
22201842



Wasi Farabi
22201358

Sadat Shahriar Arko
22201992

Shayonton Hasan
22201763

Approval

The thesis titled "A Heterogeneous Federated Learning Approach for Decentralized Pest Identification under Non-IID Distributions" submitted by

1. Md. Rafiul Islam (22201842)
2. Wasi Farabi (22201358)
3. Sadat Shahriar Arko (22201992)
4. Shayonton Hasan (22201763)

of Fall, 2022 has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science in Spring 2026.

Examining Committee:

Supervisor:
(Member)

Mr. Md. Tanzim Reza

Senior Lecturer
Department of Computer Science and Engineering
Brac University

Co-Supervisor:
(Member)

Dr. Amitabha Chakrabarty

Professor
Department of Computer Science and Engineering
Brac University

Thesis Coordinator:
(Member)

Dr. Md. Golam Rabiul Alam

Professor
Department of Computer Science and Engineering
Brac University

Head of Department:
(Chair)

Dr. Sadia Hamid Kazi

Associate Professor & Chairperson
Department of Computer Science and Engineering
Brac University

Ethics Statement

There are no human subjects, animal subjects or personal or sensitive data collected in this research. The entire experiments are performed on the IP102 benchmark dataset, which is a public set of agricultural insect pest images provided for academic research. There is no personally identifiable information, no biometric information and no content that needs informed consent or institutional ethical approval.
The main focus of this research is the preservation of the farm-level data sovereignty in federated learning deployments. This study deliberately shows a known privacy vulnerability, gradient-based property inference attacks, in a controlled and contained experimental setting, the homogeneous FL experiments. These experiments are done with simulated clients and with only the public IP102 dataset and without the involvement of real farms, real stakeholders or proprietary agricultural data. The attack is only used to measure risk and inspire the suggested architectural solution, and is not intended to be an adversarial tool for external use.
The proposed heterogeneous FL framework is designed following the concept of privacy by design. The system is structural in nature, as it avoids leakage of the gradient (no leakage is injected into the system) and offers a structural privacy guarantee without the need to create mechanisms that could become attack surfaces themselves. All code, model weights, and experimental configurations are reported so that they can be reproduced and/or independently verified.
There are no conflicts of interest. The use of the IP102 data set is subject to the conditions of access published. All quotations are properly credited.

Abstract

Federated Learning (FL) enables decentralized clients to collaboratively train a shared model without exchanging raw data, but its effectiveness is often limited by non-IID data distributions and vulnerability to gradient-based privacy attacks. This thesis proposes a heterogeneous FL framework for fine-grained agricultural pest classification on the IP102 dataset under a Dirichlet partition ($\alpha = 0.5$). The framework employs diverse client architectures and aggregates only a shared classifier head, preventing direct gradient comparison across clients. To improve learning under data heterogeneity, a server-maintained clustered spatial buffer is constructed using Hierarchical Agglomerative Clustering and used to generate class-prior knowledge for server-anchored knowledge distillation. The distillation process is progressively strengthened throughout training while preserving client data privacy, as buffer images remain exclusively on the server. Experimental results show that the proposed approach achieves 63.76% test accuracy and 0.611 F1-score, outperforming the heterogeneous baseline by 4.16 percentage points. Furthermore, the framework reduces property inference attack accuracy from 56.7%-61.7% in a homogeneous FL baseline to 0%, demonstrating strong privacy protection through architectural heterogeneity. These findings indicate that server-anchored knowledge distillation can effectively mitigate client divergence, improve classification performance under non-IID conditions, and enhance privacy without relying on differential privacy or secure multi-party computation.

Keywords: Federated Learning, Heterogeneous Networks, Non-IID Data, Gradient-Based Privacy Attacks, Architectural Incommensurability, Partial Model Aggregation, Hierarchical Agglomerative Clustering, Clustered Spatial Buffer, Class-Prior Knowledge Distillation, Agricultural Pest Identification.

Dedication

To our families, whose silent sacrifices were the reason this is made possible. We would like to thank our supervisors Mr. Md. Tanzim Reza and Dr. Amitabha Chakrabarty. For their guidance, patience and trust. To all the farmers that work on the ground, but do not receive recognition - *this work is, in some small way, for them.*

Acknowledgement

First and foremost, all praise to the Almighty God, through whose grace our thesis was completed without any major interruptions. Secondly, we extend our sincere gratitude to our supervisor, Mr. Md. Tanzim Reza sir, and co-supervisor, Dr. Amitabha Chakrabarty sir, for their kind support and guidance throughout our work. They were always available whenever we needed assistance. And finally, we are deeply grateful to our parents, without whose constant support and encouragement this achievement would not have been possible. Through their unwavering support and prayers, we are now on the verge of completing our graduation.

Table of Contents

List of Figures

List of Tables

Nomenclature

The next list describes several symbols & abbreviation that will be later used within the body of the document

α	Dirichlet concentration parameter controlling non-IID skew ($\alpha = 0.5$)
ΔW_i	Gradient update vector of client i (local minus global weights)
ϵ	Differential privacy budget parameter
γ	Non-buffer cross-entropy amplification factor ($\gamma = 1.5$)
$\hat{\mathbf{y}}$	Predicted class logit vector of dimension 102
λ	KD weight coefficient, ramped from $\lambda_{\min} = 0.30$ to $\lambda_{\max} = 0.90$
$\mathbf{h}$	Raw backbone feature vector of dimension d
$\mathbf{W}$	Classifier head weight matrix, $\mathbf{W} \in \mathbb{R}^{102\times256}$
$\mathbf{z}$	L2-normalised 256-dimensional embedding from the ProjectionMLP
$\mathcal{B}$	Server-retained clustered buffer (7,418 samples, 32 subclasses)
$\mathcal{S}_{\text{buf}}$	Set of 32 buffer subclasses selected by HAC
$\mathcal{S}_{\text{non}}$	Set of 70 non-buffer subclasses absent from $\mathcal{B}$
μ	FedProx proximal term coefficient ($\mu = 0.01$)
τ	HAC cosine distance cut threshold ($\tau = 0.10$)
d	Backbone output feature dimension (768–2048, architecture-dependent)
D_i	Private training dataset of client i
n_k	Number of private training samples at client k
$p_s^{(t)}$	Class-prior soft-label vector for buffer subclass s at round t
R	Total number of federation rounds ($R = 100$)
T	KD temperature, annealed from $T_{\max} = 4.0$ to $T_{\min} = 2.5$
ADMM	Alternating Direction Method of Multipliers
AI	Artificial Intelligence

BN Batch Normalisation

CE Cross-Entropy loss

CNN Convolutional Neural Network

CUDA Compute Unified Device Architecture

DP Differential Privacy

EMD Earth Mover's Distance

FedAvg Federated Averaging

FedProx Federated Proximal regularisation

FID Fréchet Inception Distance

FL Federated Learning

GAN Generative Adversarial Network

GAP Global Average Pooling

GPU Graphics Processing Unit

HAC Hierarchical Agglomerative Clustering

IID Independent and Identically Distributed

IoT Internet of Things

IP102 Insect Pest dataset with 102 fine-grained pest subclasses

KD Knowledge Distillation

KL Kullback–Leibler divergence

MLP Multi-Layer Perceptron

non-IID Non-Independent and Identically Distributed

RGB Red–Green–Blue colour space

VAE Variational Autoencoder

VRAM Video Random Access Memory

Chapter 1

Introduction

1.1 Background

The rapid proliferation of edge devices, distributed sensors, and privacy regulations has fundamentally transformed how machine learning systems must be designed and deployed. Federated Learning (FL) has emerged as a leading paradigm that enables multiple clients to collaboratively train a shared model without transferring raw data to a central server [10]. By keeping data local and sharing only model updates, FL satisfies the core requirements of privacy-sensitive domains such as healthcare, agriculture, and Internet of Things applications, where centralised data collection is either legally restricted or operationally infeasible.

The canonical FL algorithm, FedAvg, aggregates model parameters from all participating clients through a weighted average proportional to their local dataset sizes. While effective under idealised conditions, FedAvg assumes that all clients use the same model architecture - an assumption that rarely holds in real-world deployments. As documented by Wu et al. in the FIARSE framework, clients naturally exhibit heterogeneous computational capabilities and deploy architecturally distinct models, rendering direct parameter averaging undefined across different backbone networks [24]. This architectural incompatibility is not merely a technical inconvenience but a fundamental barrier to the scalability of federated systems.

Compounding this challenge is the problem of statistical heterogeneity. In practice, client datasets follow non-IID (non-independent and identically distributed) distributions - different clients hold data from different classes, domains, or collection conditions. Vieira demonstrated that non-IID distributions cause client gradient updates to point in conflicting directions, slowing convergence and degrading the quality of the aggregated global model [22]. Chadha et al. quantified this degradation empirically, finding that FedAvg suffers an average accuracy reduction of 27.1% under realistic non-IID partitioning compared to IID conditions [13]. These findings underline that non-IID data is not an edge case but the norm in distributed deployments.

Beyond statistical and architectural heterogeneity, privacy concerns extend deeper than the absence of raw data sharing. Luo et al. demonstrated that gradient updates shared in standard FL are vulnerable to inference attacks, through which an adversary can reconstruct statistical properties of a client's private training data [17]. Xu et al. further showed that these vulnerabilities extend to attribute reconstruction attacks, where the server or a passive adversary can infer sensitive properties from

shared model updates even without accessing raw data [25]. These findings establish that homogeneous FL - where all clients share the same parameter space - inherently exposes client data distributions through gradient similarity.
This thesis addresses these intertwined challenges in the context of agricultural pest classification, a domain characterised by sensitive farm-level data, hardware-constrained edge deployments, and severe class imbalance. We focus on the IP102 benchmark - a large-scale dataset of 102 pest species across 8 crop superclasses - and propose a single-phase heterogeneous federated learning framework that simultaneously advances classification accuracy, eliminates gradient-based privacy attacks through architectural incommensurability, and bridges knowledge across heterogeneous clients through server-anchored Knowledge Distillation.

1.2 Rationale of the Study and Motivation

The primary motivation of this work stems from three observations that together define an unsolved problem in practical federated learning.
First, existing federated learning systems impose architectural uniformity that is incompatible with real-world agricultural deployments. Farms, research stations, and agricultural monitoring services operate heterogeneous hardware ranging from GPU-equipped servers to resource-constrained embedded devices. Requiring every participant to deploy an identical backbone network - as assumed by standard FedAvg - excludes smaller participants and fails to exploit the rich diversity of visual representations that different architectures can extract. As Guo et al. documented through the PracMHBench benchmark, real edge devices vary dramatically in computational power, memory, and communication bandwidth, making a one-architecture-fits-all approach fundamentally impractical [26].
Second, the privacy guarantee of standard federated learning is weaker than commonly assumed. While raw images never leave a client, gradient updates in homogeneous FL are in the same high-dimensional parameter space for all clients. This shared geometry enables a server - or any passive observer receiving gradient updates - to apply property inference attacks that identify the statistical properties of each client's training data. In an agricultural context, knowing which crop type dominates a farm's training data reveals commercially sensitive information: crop specialisation, geographic location, and pest vulnerability profile. This constitutes a privacy breach even in the complete absence of raw data sharing.
Third, knowledge transfer across heterogeneous architectures without public data is an open problem. Data-free knowledge distillation approaches studied by Zhu et al. and Wang et al. rely on either publicly available datasets or strong teacher model assumptions that conflict with the privacy-preserving intent of federated learning [7], [23]. A principled mechanism for transferring inter-class knowledge across architecturally diverse clients - using only a small, carefully curated server-side reference set - is required.
These three motivations converge on a single research question: can a federated learning system simultaneously support heterogeneous client architectures, eliminate gradient-based privacy leakage through structural design, and effectively transfer knowledge across clients to achieve competitive pest classification accuracy?

1.3 Problem Statement

Despite significant progress in federated learning research, existing approaches fail to simultaneously address the intertwined challenges of architectural heterogeneity, data heterogeneity, and gradient-based privacy leakage in a unified framework.

The fundamental technical barrier is that parameter averaging - the basis of FedAvg - requires all clients to share identical model architectures. When clients deploy EfficientNet-B0, ResNet-50, MobileNetV3-Large, ConvNeXt-Tiny, DenseNet-121, and ConvNeXt-Small simultaneously, their weight tensors have incompatible shapes and semantics. Averaging parameters from architecturally distinct models is mathematically undefined and produces meaningless aggregations. This incompatibility prevents architectural diversity while also inadvertently providing a privacy benefit: if gradient updates have different dimensions across clients, direct comparison becomes infeasible. However, existing frameworks that enforce architectural uniformity to enable aggregation sacrifice this structural privacy property.

Data heterogeneity introduces a second challenge. Under realistic Dirichlet partitioning with concentration parameter $\alpha = 0.5$, individual clients become dominant in specific crop superclasses - one client may hold 57% Mango images while another holds 51% Corn images. Without a mechanism to align client representations around a shared reference point, locally trained models diverge severely, and the global model struggles to generalise across all 102 pest subclasses. This divergence is particularly acute for rare classes: subclasses with few training examples across all clients receive inadequate gradient signals, collapsing to near-zero accuracy.

The privacy vulnerability of homogeneous FL adds a third dimension to the problem. When all clients use the same backbone, the server receives gradient updates $\Delta W_i = W_{\text{local},i} - W_{\text{global}}$ that all reside in the same D-dimensional parameter space. A property inference attack can compute the cosine similarity between each client's update and class-specific reference gradients to identify the dominant crop type in that client's training data - without accessing a single raw image. This leakage is not theoretical: as we demonstrate empirically, such an attack achieves 56.7-61.7% accuracy (4.5-4.9× above the 12.5% random baseline) across three different homogeneous backbones.

The absence of an integrated framework that resolves all three challenges simultaneously - architectural flexibility, knowledge transfer under non-IID conditions, and structural privacy guarantees - constitutes the core gap that this thesis addresses [10].

1.4 Objectives

This thesis develops a single-phase heterogeneous federated learning framework for fine-grained agricultural pest classification that achieves the following specific objectives:

- **Demonstrate gradient-based privacy leakage in homogeneous FL:** Design and implement a property inference attack that, given only the gradient updates shared during standard FedAvg, identifies each client's dominant crop superclass. Establish the attack accuracy across multiple backbone architectures and compare against the random baseline to quantify the privacy

risk.

- **Eliminate gradient leakage through architectural incommensurability:** Propose a heterogeneous FL framework where six clients deploy structurally distinct backbone networks. By ensuring that client gradient updates reside in incompatible parameter spaces, render property inference attacks mathematically infeasible without relying on differential privacy noise or cryptographic mechanisms.

- **Enable knowledge transfer across heterogeneous architectures:** Construct a spatially diverse server-side buffer through Hierarchical Agglomerative Clustering (HAC) on visual feature centroids, ensuring that the 32 selected buffer subclasses cover all 18 distinct visual clusters in the IP102 feature space. Use this buffer to generate class-prior soft labels that bridge knowledge across client architectures each federation round.

- **Address non-IID divergence through server-anchored Knowledge Distillation:** Implement a class-prior KD mechanism with progressive lambda ramping ($\lambda : 0.30 \rightarrow 0.90$) and temperature annealing ($T : 4.0 \rightarrow 2.5$) that stabilises client training under Dirichlet-partitioned non-IID data without sharing any client data with the server or other clients.

- **Validate that buffer knowledge generalises beyond buffer subclasses:** Demonstrate empirically that server-anchored KD improves accuracy on the 70 non-buffer subclasses substantially more than on the 32 buffer subclasses, confirming genuine cross-class coordination rather than buffer memorisation.

- **Evaluate the privacy-accuracy tradeoff:** Compare classification accuracy and property inference attack accuracy across homogeneous FL, heterogeneous FL without buffer, and heterogeneous FL with server-anchored KD, quantifying the cost and benefit of each design decision.

1.5 Methodology in Brief

The proposed framework operates in three phases: offline buffer construction, server pretraining, and federated learning with server-anchored Knowledge Distillation.
Buffer Construction: Prior to any federation, we construct a spatially diverse reference buffer from the IP102 training set. Using a pretrained ConvNeXt-Small backbone, we extract 768-dimensional feature centroids for all 102 subclasses and compute a 102×102 cosine distance matrix. Hierarchical Agglomerative Clustering (HAC) with average linkage and threshold $\tau = 0.10$ partitions the 102 subclasses into 18 visually distinct clusters. From each cluster, we select 1-4 representative subclasses proportional to cluster size, resulting in 32 buffer subclasses. A 20% random sample is drawn from each selected subclass, yielding 7,418 buffer images (16.4% of the training set). These indices are strictly excluded from all client Dirichlet partitions, guaranteeing zero overlap between the server buffer and any client's private data.
Server Pretraining: The server backbone (EfficientNet-B2, selected empirically as the highest-performing architecture on buffer validation data) is pretrained for 50

epochs on the 7,418 buffer samples using cross-entropy loss with label smoothing. This establishes the server as an informed teacher before federation begins. The server's test accuracy on the 32 buffer subclasses (44.16%) serves as the comparison baseline that the federated ensemble must exceed to validate the contribution of federation.

Federated Learning: Six clients (EfficientNet-B0, ResNet-50, MobileNetV3-Large, ConvNeXt-Tiny, DenseNet-121, ConvNeXt-Small) train exclusively on their private Dirichlet-partitioned data. Each client maps backbone features through a shared projection architecture (ProjectionMLP: feat_dim $\rightarrow$ 512 $\rightarrow$ 256, L2 normalisation) to a common 256-dimensional embedding space, followed by a shared Linear(256, 102) classifier head. Each round, the server computes class-prior soft labels by averaging its predictions over all buffer images of each buffer subclass at temperature $T(r)$, and broadcasts these 32 vectors alongside the global classifier weights. Clients apply cross-entropy loss on all private data, augmented by Knowledge Distillation using the class prior when a private image's subclass belongs to the buffer set, and amplified cross-entropy (weight 1.5$\times$) for non-buffer subclasses. FedProx regularisation ($\mu = 0.01$) prevents client drift. Only the classifier head is aggregated via FedAvg each round. After 100 rounds, the best-validation checkpoint (round 91) is evaluated on the test set.

The complete pipeline is illustrated in Figure 1.1.

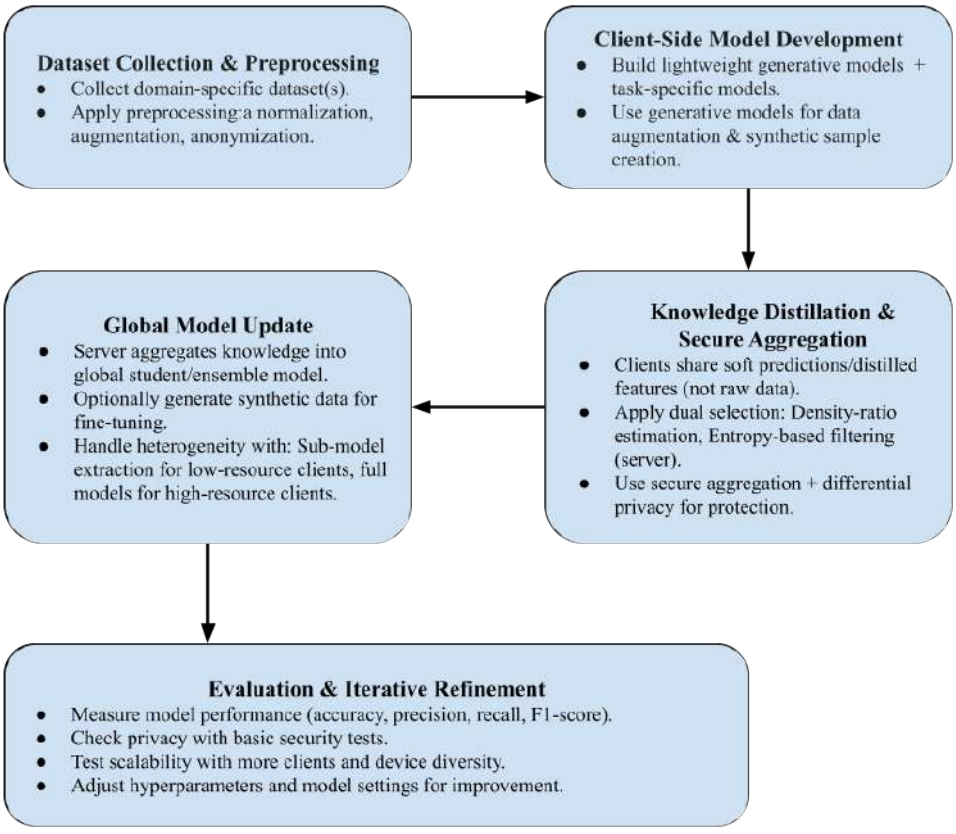


Figure 1.1: Overview of the proposed heterogeneous federated learning framework with server-anchored Knowledge Distillation.

1.6 Scope and Limitations

Scope: This work focuses on single-phase heterogeneous federated learning for fine-grained image classification under non-IID conditions, evaluated on the IP102 pest classification benchmark. The framework supports six structurally distinct backbone architectures simultaneously, makes no assumption about client hardware uniformity, and provides structural privacy guarantees through architectural incommensurability. The server buffer strategy is domain-agnostic and applicable to any multi-class visual recognition task where a small representative reference set can be constructed from publicly available or institution-held data.

Limitations: Several boundaries define the scope of the current work. First, the federation is simulated on a single machine rather than deployed across real network-connected devices; communication latency, packet loss, and straggler effects are not modelled. Second, the buffer covers only 32 of 102 subclasses, and Knowledge Distillation guidance is therefore absent for the remaining 70 subclasses during client training. Third, while the classifier-head property inference attack on heterogeneous FL achieves 0% accuracy, a sufficiently sophisticated adversary with access to multiple rounds of updates and auxiliary information may still attempt weaker forms of inference. Fourth, the privacy analysis is empirical rather than formal - no differential privacy guarantee or cryptographic proof is provided. Future work should address these limitations through asynchronous FL protocols, expanded buffer coverage strategies, and formal privacy analysis under established threat models.

Chapter 2

Literature Review

2.1 Preliminaries

Federated Learning (FL) has revolutionized the field of distributed machine learning by enabling multiple decentralized clients to collaboratively train models without exchanging raw data, thus enhancing privacy protection. This approach leverages local computational resources, allowing model parameter updates to be aggregated centrally, commonly using algorithms like Federated Averaging (FedAvg), which efficiently combines locally trained models into a global consensus model.
Extensive research has been conducted to understand FL's fundamental principles and its varied architectures. Kairouz et al. provide an exhaustive survey detailing horizontal, vertical, and federated transfer learning paradigms, highlighting the different data distribution scenarios and collaboration settings that FL can accommodate [6]. Li et al. contribute a comprehensive review of the challenges in federated learning, particularly emphasizing the issues posed by system heterogeneity, such as varying client hardware capabilities, data heterogeneity including non-IID distributions, and communication constraints [4].Additionally, Bonawitz et al. explore the practical aspects and implementation challenges of scalable FL systems, offering insights into privacy mechanisms and real world federated system deployments [1].
However, real world federated learning deployments face significant challenges that traditional approaches fail to address adequately. Model heterogeneity occurs when clients employ different neural network architectures (ResNet, VGG, MobileNet, EfficientNet) due to varying computational capabilities, hardware constraints, or task specific requirements. System heterogeneity manifests through differences in computational power, memory capacity, communication bandwidth, and availability patterns across participating devices. Data heterogeneity (non-IID data distributions) represents another fundamental challenge where client datasets exhibit statistical differences in feature distributions, class imbalances, or sample sizes, often modeled using Dirichlet distributions or Earth Mover's Distance (EMD) metrics.
Knowledge distillation has emerged as a promising solution for addressing these heterogeneity challenges. Unlike traditional parameter aggregation, knowledge distillation transfers knowledge through soft predictions (logits) or intermediate representations, enabling collaboration between models with different architectures. Qin et al. provide a comprehensive categorization of KD approaches in FL, including logit and feature based, and data free distillation methods [18]. Privacy-preserving mechanisms in federated learning include differential privacy, secure aggregation,

and synthetic data generation to protect sensitive client information while maintaining model performance.

2.2 Review of Existing Research

2.2.1 Foundational Federated Learning and Basic Data Heterogeneity Management

Early federated learning research focused on establishing fundamental aggregation mechanisms and addressing basic data heterogeneity challenges. Tedjini & Mustapha demonstrated practical FL implementation in cybersecurity applications, achieving 95.9 % binary classification accuracy on the UNSW-NB15 dataset [21]. They used federated CNN architectures under data heterogeneity conditions with both IID and non-IID data distributions while comparing FedAvg and FedProx aggregation strategies. This work established the viability of FL in sensitive domains where data sharing is restricted.
Building upon basic FL principles, Vieira introduced FedWS, incorporating weight standardization to address data heterogeneity challenges where clients have varying Earth Mover's Distance (EMD) levels of non-IID distributions [22]. The approach achieved 1.3-5.0% accuracy improvements and up to 100% communication cost savings by stabilizing gradient behavior under heterogeneous conditions, demonstrating that client side normalization techniques could significantly enhance FL efficiency without requiring protocol modifications.

More sophisticated solutions had also been proposed to cope with the challenge of federated learning. Shi et al. extended mixed (3-operator) ADMM framework for federated settings, and showed that when clients with different availability patterns work on a heterogeneous infrastructure the proposed method can converge faster and be more robust to client dropouts than classical FedAvg [11]. These works set the foundation for a more complex FL optimization from a theoretical point of view.

2.2.2 Privacy-Preserving Federated Learning with Differential Privacy

The privacy preservation of FL has matured from simple secure aggregation into an adaptive method that can handle different types of heterogeneity. Piran et al. proposed adaptive differential privacy with noise scale that varies with the training process, and obtained better accuracy while preserving decentralised privacy in heterogeneous system (i.e, various computation abilities) as well as data distribution(homogeneous such as IID or non-IID) [30]. This study removed the need for central server dependencies and also maintained privacy using dynamic noise adaptation.
Privacy-preserving Fingerprinted data heterogeneity in the applications of healthcare FL benefited in particular from advances. Kerkouche et al. provided practical predictive forecasts with R^2 scores of 0.88 -0.94 in conditions for COVID-19 prediction while obeying privacy constraints across data heterogeneity from diverse epidemic patterns and reporting infrastructures in different counties [27]. The authors

demonstrated that with small to moderate privacy budgets ($\epsilon \leq 0.5$) they could still achieve accurate epidemic forecasting without sacrificing participants' privacy.
Synthetic data generation has been revealed to be a powerful privacy protection tool for addressing the challenge of heterogeneous data. Lomurno & Matteucci presented the FedKR framework for synthetic data generation instead of raw data sharing [16]. Their method obtained an accuracy gain of 4.24% on average compared to local training and it achieved good performance even over data-limited scenarios with compression rations up to 600:1, largely reducing the attack surface when dealing with heterogeneous data sets compared with parameter sharing methods.

2.2.3 Knowledge Distillation for Model Heterogeneity Management

Knowledge distillation has evolved from the basic teacher-student paradigm to complex selection of model heterogeneity. Shao et al. addressed the fundamental limitation of requiring strong teacher models through their Selective FD framework [19]. This approach introduced a dual-level selection mechanism using density-ratio estimation on clients and entropy-based filtering on servers, achieving accuracy gains of 4.2% on MNIST, 8.3% on Fashion-MNIST, and 12.1% on CIFAR-10 while reducing communication costs by 50 to 70% under model heterogeneity conditions where clients used different CNN and MLP architectures.
Data-free knowledge distillation represented a significant advancement in model heterogeneity management. Zhu et al. pioneered data-free approaches by introducing server-side lightweight generators that produce synthetic samples for knowledge extraction from heterogeneous client models [7]. Their approach achieved 85.2% accuracy on CIFAR-10 and 65.8% on CIFAR-100, demonstrating comparable performance to homogeneous baselines while supporting diverse architectures like ResNet and VGG without requiring shared datasets under model heterogeneity scenarios.
Feature-level knowledge distillation emerged as superior to logit-based approaches for model heterogeneity. Li et al. demonstrated that feature distillation using orthogonal projection techniques outperforms logit distillation, achieving up to 16.09% higher test accuracy compared to state of the art methods and requiring only 10 rounds versus 12 rounds for HeteroFL to reach 70% accuracy on CIFAR-10 [28]. Their work established feature distillation as the preferred knowledge transfer mechanism for heterogeneous architectures under model heterogeneity conditions.
Serverless and uncertainty-aware knowledge distillation expanded FL's applicability under model heterogeneity. Chadha et al. demonstrated that KD - based aggregation could handle diverse architectures in serverless FL, improving efficiency and reliability while achieving 3.5x speedups for FedMD and 1.76x for FedDF under model heterogeneity where clients used different neural network architectures [13]. Wang et al. eliminated the need for public datasets through their FedType framework, achieving 88.5% accuracy on CIFAR-10 and outperforming prior heterogeneous FL approaches by 5-12% under model heterogeneity using proxy models to bridge different architecturess [23].

2.2.4 Computational System Heterogeneity and Resource-Aware Solutions

Resource aware FL solutions addressed the practical reality of diverse client capabilities under system heterogeneity. Wu et al. developed importance aware submodel extraction mechanisms, ranking parameters by magnitude to enable less capable clients to train smaller submodels while achieving 95% of full-model performance with only 25% of parameters [24]. The approach scaled successfully to 200 heterogeneous clients with 50% availability, demonstrating practical robustness under system heterogeneity conditions where clients had varying computational capacities.
Nested architecture approaches provided scalable solutions for system heterogeneity. Wang et al. proposed nested sub-model architectures proportional to client capacities, reducing computational costs by up to 50% on resource-limited clients while maintaining comparable accuracy to standard FL models on CIFAR-10, CIFAR-100, and ImageNet datasets [5]. This work established the foundation for resource-proportional FL participation under system heterogeneity.
Edge computing constraints drove specialized knowledge transfer mechanisms addressing system heterogeneity. He et al. addressed edge computing limitations through alternating minimization between small edge CNNs (ResNet-8 with 11K parameters) and large server models (ResNet-55/109 with 591K-1.15M parameters) [3]. Their FedGKT framework achieved 9-17 times less computational power requirements and 54-105 times fewer parameters in edge CNNs while maintaining competitive accuracy, enabling large scale model deployment on resource constrained devices under system heterogeneity.

2.2.5 Advanced Model and System Heterogeneity Management

Comprehensive benchmarking provided insights into heterogeneity management effectiveness under multiple heterogeneity types. Guo et al. established the first systematic benchmarking platform for model heterogeneity with three levels: width (different channel sizes), depth (different layer numbers), and topology (different architectures like ResNet, MobileNet, ALBERT) [26]. Their comprehensive evaluation revealed that depth-based methods often achieve superior performance across computation and communication constraints under system heterogeneity, while width-based heterogeneity suffers less degradation with increasing client numbers.
Trustworthy heterogeneous FL addressed security concerns in diverse environments under model heterogeneity. Chen et al. introduced adaptive distillation to detect and mitigate malicious updates in heterogeneous environments where clients used different CNN architectures [14]. Their framework combined malicious client detection, high quality knowledge fusion, and adaptive knowledge distillation to maintain robust performance even under adversarial conditions and model heterogeneity.
Production oriented heterogeneity solutions addressed real world deployment challenges under multiple heterogeneity types. Tan et al. provided comprehensive strategies including adaptive client selection algorithms, personalized modeling techniques, asynchronous update mechanisms, and communication compression to optimize bandwidth use under system heterogeneity, model heterogeneity, and data heterogeneity simultaneously [20].

Jin et al. introduced unified frameworks that combine privacy guarantees with support for model heterogeneity, system heterogeneity, and data heterogeneity through cryptographic secure aggregation protocols with personalized local adaptation strategies [10]. Their approach enables clients to maintain unique model architectures while handling non-IID data distributions and varying computational capabilities.

2.2.6 Generative Model Heterogeneous Federated Learning

Federated generative modeling represented the frontier of heterogeneous FL research under model heterogeneity. Peng et al. enabled federated training of diffusion models, maintaining generation quality close to centralized baselines with FID scores within 5% of centralized performance [29]. Their FedDDPM framework incorporated auxiliary data generation and oneshot correction mechanisms to handle data heterogeneity with non-IID distributions while preserving privacy under system heterogeneity constraints.

Variational approaches established foundational generative FL capabilities under model heterogeneity. Dugdale et al. provided initial exploration of federated VAE training, demonstrating feasibility on MNIST with 4-client setups using standard VAE architectures with β-VAE loss formulation [9]. Although limited in its form, this work laid a foundation for more advanced federated generative modeling techniques under model heterogeneity.

Federated learning with advanced GANs presented sophisticated privacy generation trade-offs in the presence of model heterogeneity. Wijesinghe et al. used the selective GAN component sharing to reduce 40-60% communication overhead and obtain the competitive generation quality with comparable FID score compared with centralized training [12]. This model heterogeneity problem was dealt with by enabling the client to select a different GAN architecture, but share only parts of it.

Kim et al. proposed cross-client adaptive re-weighting for generative modeling, where they estimated the FID score of generated samples on each client and adapted the weight proportionally at aggregation stage in presence of data heterogeneity using non-IID chest X-ray datasets [15]. Very remarkably, Poquito's federated training with FedCAR even beats centralized learning in some tasks, which implies the need of high quality crowd performances for hybrid aggregation.

In an in-depth privacy protective generative framework, most methods of various types of heterogeneity are used. Xu et al. introduced FLiP which trains on samples using dataset distillation compressed by 600:1 factor, but maintains the same model performance even when the data is non-homogenous [25]. An attack on privacy test used attacks which seek unrelated attributes, and was found that 7 out of 12 attacks did slightly better than random guessing namely, 50% accuracy. This indicates that privacy remains safeguarded by the method, even in varying model environments.

2.3 Summary of Key Findings

The literature of the recent past shows a paradigm shift in Federated Learning (FL) toward a holistic framework that aims to address the triple challenge of heterogeneity across models, systems, and data. Moving beyond simple weight aggregation to more advanced knowledge transfer and adaptive architectures, these developments enable FL to support a wide range of hardware ecosystems without compromising privacy or performance. Table 2.1 summarises the principal findings across six theme areas.

Table 2.1: Summary of key findings in federated learning research.

Theme	Key Finding
Model Heterogeneity	Knowledge distillation (KD) is the most effective solution for model-heterogeneous FL, with feature-based distillation significantly outperforming logit-based methods. Selective Feature Buffering can reduce communication costs by up to one-half to one-third without degrading model quality.
System Optimisation	Discriminative sub-model analysis achieves approximately 95% of baseline performance while using only one quarter of the parameters. Resource-constrained devices reduce computational overhead by up to 50% through proportional model splitting and efficient neural architectures.
Data Heterogeneity (Non-IID)	Selective sharing and adaptive clustering strategies have replaced conventional normalisation techniques, resulting in faster convergence and improved classification accuracy on non-IID datasets.
Generative Capabilities	Federated generative adversarial networks can achieve performance comparable to centralised training at only 40-60% of the computational cost. Adaptive reweighting schemes further enable certain federated models to outperform centralised counterparts by leveraging diverse, high-quality data.
Privacy & Compression	Synthetic data generation allows data compression by up to 600-fold without violating privacy constraints. However, a trade-off remains between the magnitude of differential privacy noise and model fidelity.
Remaining Gaps	Current research lacks formalised mathematical privacy guarantees, large-scale real-world evaluations involving thousands of heterogeneous clients, and a unified benchmarking framework that jointly addresses model, system, and data heterogeneity.

Chapter 3

Requirements, Impacts and Constraints

3.1 Final Specifications and Requirements

To deploy the proposed Heterogeneous Federated Learning (FL) framework successfully and evaluate it, specific configurations were established across data management, architectural design, and computational infrastructure. This section describes the methodological and technical requirements governing the experimental environment, ensuring reproducibility and validity in the evaluation of both agricultural pest classification performance and gradient-leakage prevention.

3.1.1 Data and Partitioning Specifications

The framework is based on the IP102 benchmark dataset, comprising 45,095 training samples, 7,508 validation samples, and 22,619 test samples across 102 fine-grained pest subclasses. These subclasses are mapped to eight biological superclasses - Rice, Corn, Wheat, Beet, Alfalfa, Vitis, Citrus, and Mango - to enable property inference attack evaluation at the crop-host level.
Private client data is partitioned using a Dirichlet distribution with concentration parameter $\alpha = 0.5$, applied at the superclass level to simulate the non-IID data skew typical of decentralised agricultural networks.
A globally shared buffer is constructed offline using Hierarchical Agglomerative Clustering (HAC) applied to per-subclass centroids extracted from ConvNeXt-Small ImageNet features. Average linkage with a distance threshold of $\tau = 0.10$ produced 18 clusters, from which 32 representative subclasses were selected, yielding a 7,418-sample clustered buffer. This buffer is retained exclusively at the server throughout federated training. Clients receive no buffer images. Each federation round, the server passes the buffer through its pretrained backbone to generate per-class soft-label priors, which are broadcast to clients as knowledge distillation targets for their locally held buffer-subclass samples.

3.1.2 Architectural and Methodological Requirements

The system requires a strictly heterogeneous ensemble to demonstrate that architectural diversity acts as a structural deterrent to property inference attacks. Prelimi-

nary buffer-test accuracy was used to rank seven vision architectures. EfficientNet-B2, which achieved the highest buffer test accuracy (81.72%), is designated as the central server. The remaining six architectures - EfficientNet-B0, ResNet-50, MobileNetV3-Large, ConvNeXt-Tiny, DenseNet-121, and ConvNeXt-Small - serve as the heterogeneous clients.
Every node uses a consistent `FLClientModel` wrapper. Raw backbone features of varying dimensionality are passed through a standardised `ProjectionMLP` (hidden dimension 512, projection dimension 256, projection dropout 0.5) followed by L2 normalisation, producing a shared embedding space. A linear classifier head maps the 256-dimensional embedding to 102 subclasses (classifier dropout 0.3).
The federated training protocol requires:

- **Loss function:** Each local training step combines three components. For private images belonging to buffer subclasses, the loss is Cross-Entropy (CE) plus a class-prior Knowledge Distillation (KD) term weighted by λ_{kd}, where the teacher signal is the server-generated per-class soft-label prior at the current distillation temperature T. Private images of non-buffer subclasses receive CE only, amplified by a factor of 1.5 to prevent classifier bias toward the KD-supervised buffer classes. A FedProx proximal regularisation term ($\mu = 0.01$) is appended to penalise classifier weights that deviate from the globally broadcast state. The KD weight ramps linearly from $\lambda_{\text{kd}} = 0.30$ to 0.90 over 100 rounds, and the distillation temperature anneals from $T = 4.0$ to 2.5.

- **Aggregation constraint:** The FedAvg algorithm aggregates only the shared 256×102 classifier head (26,112 parameters). Backbone and projection parameters are never transmitted. The physical dimensional mismatch across heterogeneous backbones - ranging from 4.2 M (MobileNetV3-Large) to 49.5 M (ConvNeXt-Small) parameters - renders gradient-based cosine-similarity attacks mathematically undefined.

3.1.3 Hardware and Software Infrastructure

Simultaneous training of seven deep learning architectures requires high-performance hardware.

- **Hardware:** The simulation is executed on a single workstation equipped with an NVIDIA GeForce RTX 4080 SUPER GPU (16 GB VRAM). This prevents out-of-memory errors when instantiating the heavier models (e.g., ConvNeXt-Small with 49.5 M parameters) alongside the server model.

- **Software configuration:** The environment runs on Windows with a dedicated Python virtual environment (`.venv`). DataLoaders are configured with `batch_size = 128`, 4 persistent background workers, and `pin_memory = True` to maximise throughput and avoid the process-spawning overhead native to Windows.

3.2 Societal Impact

The application of AI to agriculture carries consequences that extend well beyond technical performance metrics. This research introduces a Heterogeneous Federated

Learning framework for pest classification that addresses agricultural sustainability, data privacy, and equitable access to technology simultaneously.

3.2.1 Societal and Cultural Aspects

Agriculture is a community-driven practice. Traditional centralised AI models - and standard homogeneous FL - typically require uniform hardware and centralised data pooling, which marginalises resource-constrained communities and compromises data sovereignty.
By supporting architectural heterogeneity, this framework enables a resource-limited node running a lightweight MobileNetV3-Large (4.2 M parameters) to collaborate on equal terms with a resource-rich node running ConvNeXt-Small (49.5 M parameters). This lowers the financial barrier of hardware uniformity and shifts the paradigm from corporate data extraction to collaborative local intelligence, allowing regional agricultural organisations to retain full ownership of their ecological data while contributing to a shared classification head.

3.2.2 Health and Safety Aspects

Rapid and accurate identification of agricultural pests is critical for food security. The IP102 dataset covers 102 pest subclasses affecting eight major global crops - Rice, Corn, Wheat, Beet, Alfalfa, Vitis, Citrus, and Mango - that constitute fundamental dietary staples worldwide.
When pests are misidentified or detected late, the default response is broad-spectrum chemical pesticide application. This framework supports precision agriculture by enabling accurate edge-deployable classification at 63.76% macro test accuracy over 102 fine-grained classes. Fast, localised identification allows for targeted interventions, reducing chemical runoff into water supplies, minimising toxic exposure for agricultural workers, and protecting beneficial insect populations essential to local ecosystems.

3.2.3 Legal and Privacy Aspects

Agricultural data - including crop yields, proprietary pest vulnerability maps, and localised genotype information - is commercially sensitive. Unauthorised exposure can lead to market manipulation and corporate espionage.
The homogeneous FL experiments in this study demonstrate that standard FL protocols are vulnerable to gradient-based Property Inference Attacks. A malicious server could infer the dominant crop-host superclass of a private client with 61.7% accuracy - nearly 4.9× the 12.5% random baseline - constituting a severe data-sovereignty violation. By enforcing architectural heterogeneity and restricting aggregation to the shared 256-to-102 classifier head, this framework renders such attacks mathematically undefined. Backbone gradient update vectors occupy incommensurable parameter spaces across clients, eliminating the cosine-similarity computation the attack relies upon. The result is 0% property inference attack accuracy, providing a provable privacy guarantee aligned with data-protection frameworks.

3.3 Environmental Impact and Sustainability

3.3.1 Ecological Preservation through Precision Pest Management

Conventional pest management relies on the prophylactic, broad-spectrum application of chemical pesticides, leading to groundwater contamination, soil microbiome disruption, and the loss of non-target beneficial insect populations. By providing an accurate, edge-deployable classification system, this research supports targeted, species-specific interventions rather than universal chemical agents. This minimises chemical runoff, lowers the carbon footprint associated with excess pesticide production, and preserves regional biodiversity essential to long-term soil health.

3.3.2 Computational Sustainability

By supporting architectural heterogeneity, the framework prevents the forced obsolescence of legacy or low-power edge hardware. Local nodes are not mandated to run energy-intensive large models. A lightweight MobileNetV3-Large (4.2 M parameters) can actively contribute to global aggregation alongside a ConvNeXt-Small (49.5 M parameters), drastically reducing local training energy consumption and curtailing electronic waste generated by forced infrastructure upgrades.

3.3.3 Efficient Communication

Rather than transmitting full model state dictionaries - which span hundreds of megabytes per client per round in standard homogeneous FL - the proposed system restricts communication to the shared 256×102 classifier head. The class-prior KD signal broadcast from server to clients consists of 32 soft-label vectors per round, an extremely compact payload. Consequently, network bandwidth and associated energy expenditure across 100 federated rounds remain minimal.

3.4 Ethical Issues and Professional Responsibilities

3.4.1 Data Privacy as a Fundamental Right

Localised agricultural data constitutes sensitive intellectual property. The homogeneous FL baseline experiments demonstrate that a malicious server can perform property inference attacks using the aggregated model state to infer a client's dominant crop-host superclass with up to 61.7% accuracy. Deploying such a system in production would represent a violation of professional ethics and the data sovereignty of participating communities.

The proposed Heterogeneous FL framework fulfils the professional mandate of "privacy by design" by restricting aggregation to the shared classifier head and exploiting architectural incommensurability to neutralise gradient leakage entirely, achieving 0% property inference attack accuracy without any noise injection.

3.4.2 Algorithmic Equity and the Digital Divide

Engineering ethics demand that systems promote fairness and accessibility. Requiring all FL nodes to use identical, large architectures structurally excludes organisations with limited computational resources. This framework demonstrates that lightweight edge devices (MobileNetV3-Large, 4.2 M parameters) can collaborate within the same training loop as enterprise-grade backbones (ConvNeXt-Small, 49.5 M parameters), democratising access to collaborative agricultural AI regardless of hardware capacity.

3.4.3 Ecological Ethics and Systemic Responsibility

Engineers hold a core obligation to minimise negative environmental impacts. This project addresses this on two fronts. First, accurate edge classification over 102 pest classes presents a viable alternative to broad-spectrum chemical pesticides, protecting local biodiversity and human health. Second, restricting aggregation to the terminal classifier head and limiting the KD broadcast to compact per-class soft-label vectors significantly minimises network bandwidth and total FLOPs per client, reducing the carbon footprint of each training round.

3.5 Methodological and Technical Standards

3.5.1 Benchmarking and Data Standards

The experimental design uses the IP102 Benchmark Dataset, a standardised, large-scale corpus widely used in agricultural insect pest detection research. Its existing 102-class taxonomy and hierarchical mapping to 8 superclasses allow direct comparison with centralised baselines. The Dirichlet partitioning protocol ($\alpha = 0.5$, superclass-stratified) follows standard academic practice for simulating non-IID data silos in federated settings.

3.5.2 Algorithmic and Architectural Standards

- **Deep learning architectures:** The system incorporates widely validated vision backbones - ResNet-50, EfficientNet-B0/B2, MobileNetV3-Large, DenseNet-121, and ConvNeXt-Tiny/Small - ensuring that baseline feature extraction capabilities are structurally sound and independently reproducible.

- **Federated protocols:** Global classifier synchronisation follows the Federated Averaging (FedAvg) algorithm. Client drift under non-IID partitioning is mitigated by the FedProx proximal regularisation term ($\mu = 0.01$). Knowledge transfer from the server-retained buffer to clients is implemented as class-prior Knowledge Distillation with a linearly ramped weight ($\lambda_{\text{kd}} : 0.30 \rightarrow 0.90$) and annealed temperature ($T : 4.0 \rightarrow 2.5$) over 100 federation rounds.

3.5.3 Privacy and Security Standards

The architectural design aligns with the "Privacy by Design" principle. Empirical evaluation shows that standard homogeneous FL fails this standard, with property

inference attack accuracies of 56.7–61.7% against a 12.5% random baseline. The proposed system achieves 0% attack accuracy by enforcing architectural heterogeneity and restricting the communication payload to the shared 256×102 classifier head, rendering gradient cosine-similarity attacks mathematically undefined.

3.5.4 Reproducibility Standards

All experiments are seeded with a fixed random number generator (seed = 42) applied across all clients and data samplers. A centralised configuration dictionary documents all hyperparameters - learning rates, batch sizes, weight decay, dropout rates, KD schedule parameters, and local epochs - enabling independent verification and reproduction of the reported results.

3.6 Project Management Plan

3.6.1 Project Schedule and Milestones

The project lifecycle was divided into four sequential phases:

- **Phase 1 - Data curation and buffer construction:** Processing of the IP102 dataset, Dirichlet ($\alpha = 0.5$) non-IID partitioning, ConvNeXt-Small feature extraction for all subclass centroids, and HAC (Average linkage, $\tau = 0.10$) to construct the 7,418-sample, 32-subclass clustered buffer.
- **Phase 2 - Vulnerability assessment (Homogeneous FL):** Simulation of a standard uniform FL network across identical backbone architectures. Execution of the gradient-based Property Inference Attack, establishing that homogeneous systems exhibit 56.7–61.7% privacy leakage against a 12.5% random baseline.
- **Phase 3 - Proposed system deployment (Heterogeneous FL):** Implementation of the 6-client heterogeneous network with the server-retained buffer and class-prior KD protocol. Verification that architectural incommensurability eliminates property inference attacks (0% attack accuracy). Iterative hyperparameter optimisation - KD schedule, temperature annealing, non-buffer CE amplification - achieving a final test accuracy of 63.76% (F1 = 0.611) and a 12.57 pp improvement in non-buffer class accuracy over the no-buffer baseline.
- **Phase 4 - Evaluation and documentation:** Per-subclass analysis of buffer vs. non-buffer class performance, generation of all result figures, and compilation of the final technical thesis.

3.6.2 Resource Management

- **Human resources:** Research and development was conducted by a four-member academic project group. Tasks were decoupled across data preprocessing, model architecture design, empirical evaluation, and thesis formulation, with the written workload balanced equally between members to ensure timely delivery.

- **Technical resources:** Software management centred on a clean, isolated Python virtual environment (`.venv`) on Windows. DataLoader configurations were manually tuned to maximise GPU utilisation and eliminate Windows process-spawning overhead.

3.6.3 Budgetary Constraints

- **Data costs:** Utilisation of the open-source IP102 benchmark dataset eliminated field data collection and expert annotation costs.
- **Infrastructure costs:** All experiments were executed locally on a single workstation (NVIDIA RTX 4080 SUPER, 16 GB VRAM), entirely avoiding recurring cloud computing fees.
- **Deployment economics:** Because the framework natively supports lightweight edge devices (e.g., MobileNetV3-Large), real-world deployment does not require expensive, uniform high-end hardware clusters, making it economically viable for resource-constrained agricultural cooperatives.

3.7 Risk Management and Mitigation Strategies

3.7.1 Privacy and Security Risks

Identified risk: Distributed learning frameworks are inherently vulnerable to gradient-based reverse-engineering by a malicious server. The homogeneous FL baseline confirms this: a property inference adversary achieves 61.7% accuracy at inferring a client's dominant crop-host superclass from gradient updates - nearly 4.9× the 12.5% random baseline.
Mitigation: Architectural heterogeneity is the primary countermeasure. Clients deploy diverse backbone architectures, ensuring that gradient update vectors occupy incommensurable parameter spaces. Only the shared 256×102 classifier head is aggregated. Physical dimensional mismatches make cosine-similarity-based inference attacks mathematically undefined, reducing the attack accuracy to 0%.

3.7.2 Statistical Heterogeneity and Client Drift

Identified risk: Non-IID data partitioning ($\alpha = 0.5$) causes high statistical heterogeneity, leading to client drift in which local updates diverge radically and degrade global model quality. Because clients hold private data only with no shared dataset, there is no common distribution to naturally align feature spaces.
Mitigation: Two safeguards are implemented without requiring data sharing:

- **FedProx regularisation:** A proximal term ($\mu = 0.01$) is appended to the local loss, penalising classifier weights that deviate excessively from the globally broadcast state.
- **Class-prior KD from the server buffer:** Each round, the server generates per-class soft-label priors from its retained buffer and broadcasts them to

clients. Clients apply these priors as KD targets on their locally held buffer-subclass samples, anchoring the classifier's buffer-subclass geometry against drift while leaving non-buffer subclasses free to adapt to private data.

3.7.3 Computational Constraints and Resource Exhaustion

Identified risk: Simultaneously instantiating seven deep learning architectures on a single machine introduces the risk of CUDA out-of-memory (OOM) errors, particularly with the 49.5 M parameter ConvNeXt-Small. Windows process-spawning overhead can also cause DataLoaders to become a bottleneck.
Mitigation: Memory is managed via strict gradient isolation across local training steps on the RTX 4080 SUPER (16 GB VRAM). DataLoaders are configured with `batch_size = 128`, `pin_memory = True`, and 4 persistent background workers to eliminate spawning overhead and sustain continuous GPU utilisation.

3.8 Economic Analysis and Cost-Benefit Estimation

3.8.1 Developmental and Infrastructure Costs

- **Data acquisition:** The IP102 benchmark (45,095 training, 7,508 validation, 22,619 test samples) is publicly available, eliminating field-based data gathering and expert annotation costs.
- **Compute infrastructure:** The full multi-node simulation was developed and executed on a single local workstation, completely avoiding cloud instance fees.

3.8.2 Deployment and Operational Efficiency

- **Reduced CAPEX at the edge:** Homogeneous FL forces all nodes to use large, expensive architectures. The proposed framework natively supports asymmetric deployment: a resource-limited node can run MobileNetV3-Large (4.2 M parameters) and collaborate with an enterprise-grade ConvNeXt-Small (49.5 M parameters) without hardware homogenisation.
- **Reduced OPEX in communication:** Restricting per-round communication to the 256×102 classifier head (26,112 parameters) and the 32-vector KD prior broadcast is orders of magnitude cheaper in bandwidth than transmitting full model state dictionaries each round, scaling favourably for rural networks with limited data plans.

3.8.3 Cost-Benefit Yield

- **Precision agriculture returns:** Achieving 63.76% test accuracy over 102 fine-grained pest classes enables targeted, species-specific interventions rather than broad-spectrum pesticide application, reducing procurement and environmental remediation costs for agricultural operators.

- **Privacy risk avoidance:** In the homogeneous baseline, a malicious server achieves 61.7% property inference accuracy. Data breaches in commercial agriculture can result in legal penalties, intellectual property loss, and market manipulation. The proposed framework eliminates this risk entirely (0% attack accuracy) by enforcing architectural heterogeneity, trading a 1.99 pp accuracy gap relative to the best homogeneous configuration for a provable, mechanism-free privacy guarantee.

Chapter 4

Proposed Methodology

4.1 Design Process and Methodology Overview

4.1.1 Problem Framing and Design Objectives

The major challenges that are tackled in this work are the design of a federated learning (FL) system that can learn a fine-grained agricultural pest identification task simultaneously while meeting three constraints: statistical heterogeneity arising from non-IID data distributions across clients, architectural heterogeneity arising from diverse edge device capabilities, and a structural requirement for privacy preservation without the accuracy penalty typically associated with differential privacy mechanisms. These constraints are not independent - design decisions that address one tend to worsen the others - and their co-existence on the IP102 benchmark dataset (75,222 images across 102 pest subclasses partitioned via a Dirichlet distribution with concentration parameter $\alpha = 0.5$) makes the problem substantially harder than any single-constraint formulation.
The proposed framework is designed around four objectives, listed in descending order of precedence:

1. **Privacy by design:** Privacy guarantees should be grounded in architectural properties rather than noise injection. Noise should serve only as a utility-preserving mechanism, rather than the basis of privacy protection. This avoids introducing additional gradient-level information leakage while maintaining model performance.

2. **Cross-class generalisation:** The shared knowledge mechanism should improve performance on classes not directly represented in the shared buffer, not merely on classes it directly supervises.

3. **Classifier accuracy:** The ensemble should exceed the server-only baseline on both validation and test splits, demonstrating federated benefit over centralised pretraining on limited data.

4. **Architectural flexibility:** Clients should be permitted to use heterogeneous backbone architectures to accommodate varying computational budgets, without requiring any model architecture to be disclosed to the server.

These objectives jointly rule out standard homogeneous FedAvg - which satisfies objectives 3 and 4 only partially, and fails objectives 1 and 2 - and motivate the specific design choices elaborated throughout this chapter.

4.1.2 High-Level System Design

The proposed framework proceeds through two sequential stages: an offline buffer construction phase and an online federated learning phase. Figure 4.1 presents the overall architecture.

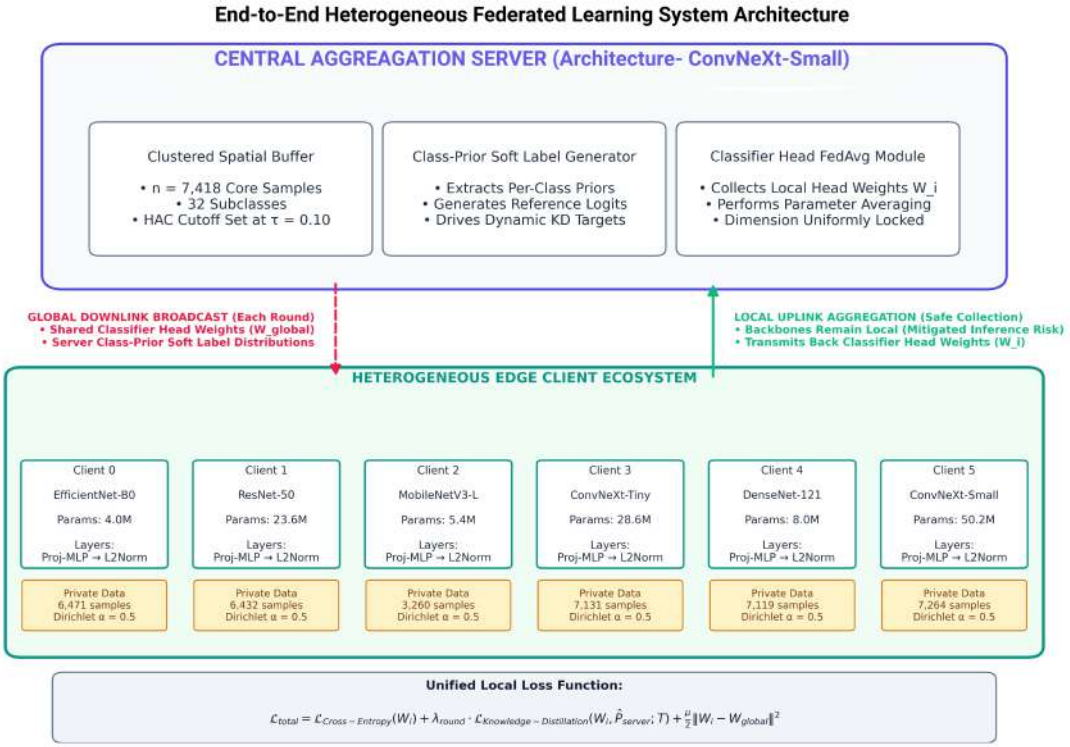


Figure 4.1: Overall architecture of the proposed heterogeneous federated learning system.

Stage I - Clustered Buffer Construction (Offline): A pre-existing reference set, called the clustered buffer, is created from the global training corpus before any federation. This is carried out once by the central server based on the feature representations extracted from the backbone with Hierarchical Agglomerative Clustering (HAC) method. A statistically representative subset of 16.4% (7,418 samples) of the 102 subclasses of pests (32 subclasses) is present in the buffer. Raw data sharing between organisations happens only with the server holding the buffer and never on the client. The construction process and its subclass selection criteria are detailed in Section 4.3.

Stage II - Heterogeneous Federated Learning (Online): The FL phase is a standard round-based coordination protocol that incorporates three modifications that collectively address the design objectives. First, clients run heterogeneous backbone architectures. This means that only a small shared classifier head (`Linear 256 → 102`) is aggregated using FedAvg, which makes it mathematically impossible to compare full model gradients across clients with different architectures. Sec-

ond, the server is pre-trained on the clustered buffer, and then sends out per-class soft-label priors - averaged softmax distributions over buffer images of each of the 32 buffer subclasses - to the clients (broadcast every round). Clients use class-prior knowledge distillation (KD) only on their private images, which are part of the buffer subclasses. Third, a non-buffer cross-entropy amplification scheme (weight 1.5×) also boosts the gradient signal for the 70 non-buffer subclasses, thereby avoiding the possibility of classifiers specialising towards the 32 KD-supervised subclasses.

4.1.3 Design Rationale

The final federated learning framework is the outcome of a number of design decisions based on privacy, efficiency and actual performance considerations. Before talking about the implementation details of the system, the motivation behind these choices is given below.

Architectural heterogeneity as a privacy mechanism: There are many reasons to use different backbones on the different clients (although edge devices also have very different compute power), but one of them is a formal property of privacy. By utilizing gradient similarity metrics, when clients share the same parameter space, the server can infer properties of individual clients' training data. This attack surface is structurally removed when parameter spaces are incommensurable. This property is quantified using a property inference attack evaluation.

Buffer at server, not at clients: To achieve this, an early design stage involved adding buffer samples to the server-side and clients-side and training private data on top of per-image KD. The accuracy in each of the 70 classes was decomposed per class and this design resulted in an improvement of about 10 percentage points in buffer-class accuracy and a degradation of about 5 percentage points in non-buffer-class accuracy, thus providing a classifier that was biased toward the 32 buffer classes at the expense of the other 70 classes. This imbalance was addressed by migrating the buffer exclusively to the server - using class-averaged soft labels as a lightweight proxy for per-image distillation - which resulted in a 12.57 percentage point improvement in non-buffer-class accuracy over the no-buffer baseline (33.14% $\rightarrow$ 45.71%). This empirical finding motivates the final architecture and is discussed in detail in Section 4.4.

Class-prior KD rather than per-image KD: Per-image knowledge distillation involves the server producing a soft label for each buffer image in each round, and then sending these labels to clients (7,418 probability vectors of dimension 102 are to be transmitted per round) or keeping buffer images at clients (the gradient crowding problem described above). Class-prior KD compresses this to 32 vectors of dimension 102 per round, only a small amount of communication is needed and still preserves the direction from server to client. The cost is that the intra-class variance of the soft labels is "smeared" during the distillation process, which is partially mitigated by the fact that the distillation temperature is reduced to 2.5 from 4.0 during the training run, sharpening the labels more and more as the client feature representations converge.

4.2 Preliminary Design and Model Specification

4.2.1 Dataset Characteristics and Class Taxonomy

All experiments are performed on the IP102 benchmark dataset, a large-scale dataset for insect pest recognition containing 75,222 images distributed across 102 fine-grained pest subclasses [2]. The dataset exhibits two characteristics that make it particularly challenging for federated learning. The numbers of samples in each subclass range from less than 50 to greater than 2,000 and a hierarchical taxonomic structure in which the 102 subclasses are organised into 8 crop-host superclasses such as Rice, Corn, Wheat, Beet, Alfalfa, Vitis, Citrus, and Mango. The authors of the dataset did the standard partitioning which resulted in 45,095 images being used for training, 7,508 in the validation set, and 22,619 for testing. All splits are class-stratified and the validation and test sets maintain a balanced-per-superclass distribution against which generalisation is measured throughout.
The hierarchical taxonomy is explicitly used in the data partitioning protocol described in Section 4.2.4, in which non-IID skew is induced at the superclass level instead of the subclass level. This option preserves realistic within-farm specialisation patterns where a client representing a rice-farming region holds predominantly rice-pest images. It ensures that the Dirichlet concentration parameter controls interpretable distributional skew.

4.2.2 Unified Client and Server Model Architecture

A single model class, `FLClientModel`, serves both the server and each client, accommodating heterogeneous backbone feature dimensions through a shared projection structure. The architecture consists of three sequentially composed components.
Backbone encoder: An ImageNet-pretrained convolutional or vision architecture that maps an input image $\mathbf{x} \in \mathbb{R}^{3\times224\times224}$ to a feature vector $\mathbf{f} \in \mathbb{R}^d$, where the feature dimension d is backbone-specific. Global average pooling is applied to spatial feature maps before the final dense representation is produced.
Projection MLP: A two-layer bottleneck network with input dimension d, hidden dimension 512, and output dimension 256. The network applies batch normalisation and ReLU activation between the two linear layers, with dropout ($p = 0.5$) applied after activation to regularise the projected representation. The final 256-dimensional embedding is L2-normalised prior to classification:

$$\mathbf{z} = \mathbf{W}_2(\text{Dropout}(\text{ReLU}(\text{BN}(\mathbf{W}_1\mathbf{h} + \mathbf{b}_1)))) + \mathbf{b}_2 \tag{4.1}$$

where the clamping prevents degenerate zero-norm representations. Even if using mixed-precision training, the projection is executed with full fp32 precision, to avoid numerical overflow in the backward pass (which is especially critical for larger backbone feature dimensions, such as 2048 in the case of ResNet-50).
Shared classifier head: A single linear layer $\mathbf{W} \in \mathbb{R}^{102\times256}$ with no bias, mapping the L2-normalised embedding to logits over the 102 pest subclasses. The normalised embedding is regularised again by a classifier dropout ($p = 0.3$) prior to the linear transformation, giving an extra regularisation on the shared parameter surface. This head is the only component exchanged between clients and the server during federated aggregation. The backbone and projection parameters remain local to each

participant all along the entire training process.
The formal forward pass is:

$$\mathbf{h} = \text{Backbone}(\mathbf{x}) \tag{4.2}$$

$$\mathbf{z} = \text{ProjectionMLP}(\mathbf{h}) \tag{4.3}$$

$$\tilde{\mathbf{z}} = \text{L2Norm}(\mathbf{z}) = \frac{\mathbf{z}}{\max(\|\mathbf{z}\|_2,\ \varepsilon)}, \quad \varepsilon = 10^{-6} \tag{4.4}$$

$$\hat{\mathbf{y}} = \mathbf{W} \cdot \text{Dropout}(\tilde{\mathbf{z}}) \tag{4.5}$$

This architecture is intentionally minimalist by confining the shared surface to a single linear layer of 26,112 parameters (256×102). So the amount of information that must traverse the network boundary is limited and the gradient signal available to a potential adversary for inference attacks is substantially compressed compared to full-model sharing.

4.2.3 Backbone Evaluation and Server Selection

Seven ImageNet-pretrained backbone architectures spanning a broad range of computational profiles are evaluated as candidate models. These are EfficientNet-B2, EfficientNet-B0, ResNet-50, MobileNetV3-Large, ConvNeXt-Tiny, DenseNet-121, and ConvNeXt-Small. Each backbone is integrated into a `FLClientModel` instance and trained independently on the clustered buffer (described in Section 4.3) as a 32 buffer-class classifier. Performance is evaluated on the held-out test split after training, with the best-performing backbone assigned the server role and the remaining six serving as heterogeneous clients.
Table 4.1 reports test accuracy on the buffer-derived training split for each backbone, along with the feature dimensionality d entering the projection MLP.

Table 4.1: Backbone ranking by buffer test accuracy. The top-ranked backbone (EfficientNet-B2) is assigned the server role; the remaining six form the heterogeneous client ensemble.

Rank	**Backbone**	**Feature dim** (d)	**Buffer test acc.**	**Role**
1	EfficientNet-B2	1,408	81.72%	Server
2	EfficientNet-B0	1,280	81.60%	Client 0
3	ResNet-50	2,048	80.44%	Client 1
4	MobileNetV3-Large	960	80.37%	Client 2
5	ConvNeXt-Tiny	768	79.02%	Client 3
6	DenseNet-121	1,024	78.80%	Client 4
7	ConvNeXt-Small	768	77.77%	Client 5

EfficientNet-B2 achieves the highest buffer test accuracy (81.72%) and is selected as the server backbone. This choice is significant not just in terms of architecture, but beyond. The server's backbone is set throughout the FL phase and determines the quality of class-prior soft labels that are sent to clients each round. Therefore, assigning the strongest backbone to the server directly maximises the utility of the knowledge distillation signal. The rest of the six backbones form a deliberately

diverse client ensemble, including lightweight mobile architectures (MobileNetV3-Large), standard deep residual networks (ResNet-50), dense connectivity patterns (DenseNet-121), and modern depthwise-separable designs (ConvNeXt-Tiny, ConvNeXt-Small), mirroring the architectural heterogeneity expected in real-world edge deployments.

4.2.4 Non-IID Data Partitioning

The partitioning is performed on client data with a Dirichlet protocol with concentration parameter $\alpha = 0.5$ (superclass level). For each of the 8 crop-host superclasses, a Dirichlet draw $\mathcal{D}_i \sim \text{Dir}(\alpha \cdot \mathbf{1}_6)$ produces a six-dimensional proportion vector governing the fraction of that superclass's samples allocated to each client. Within each given superclass, all subclasses follow the same proportional split, preserving realistic within-superclass co-occurrence patterns. Lower values of α produce more skewed distributions; $\alpha = 0.5$ is a standard choice in the FL non-IID literature that allows significant heterogeneity instead of completely eliminating minority-class representation at every client.
The buffer indices (7,418 samples spanning 32 subclasses) are not included from the partitioning pool prior to the Dirichlet split, ensuring that no sample appears in both a client's private dataset and the server buffer. This disjoint guarantee is enforced by construction where global sample indices in the buffer set are filtered out prior to superclass-level grouping, so that every sample assigned to a client is strictly private. The resulting per-client private sample counts for the proposed system are reported in Table 4.2.

Table 4.2: Per-client private sample counts after Dirichlet partitioning ($\alpha = 0.5$, buffer excluded).

Client	**Backbone**	**Private samples**
0	EfficientNet-B0	6,471
1	ResNet-50	6,432
2	MobileNetV3-Large	3,260
3	ConvNeXt-Tiny	7,131
4	DenseNet-121	7,119
5	ConvNeXt-Small	7,264
-	Total (private)	37,677
-	Server buffer	7,418

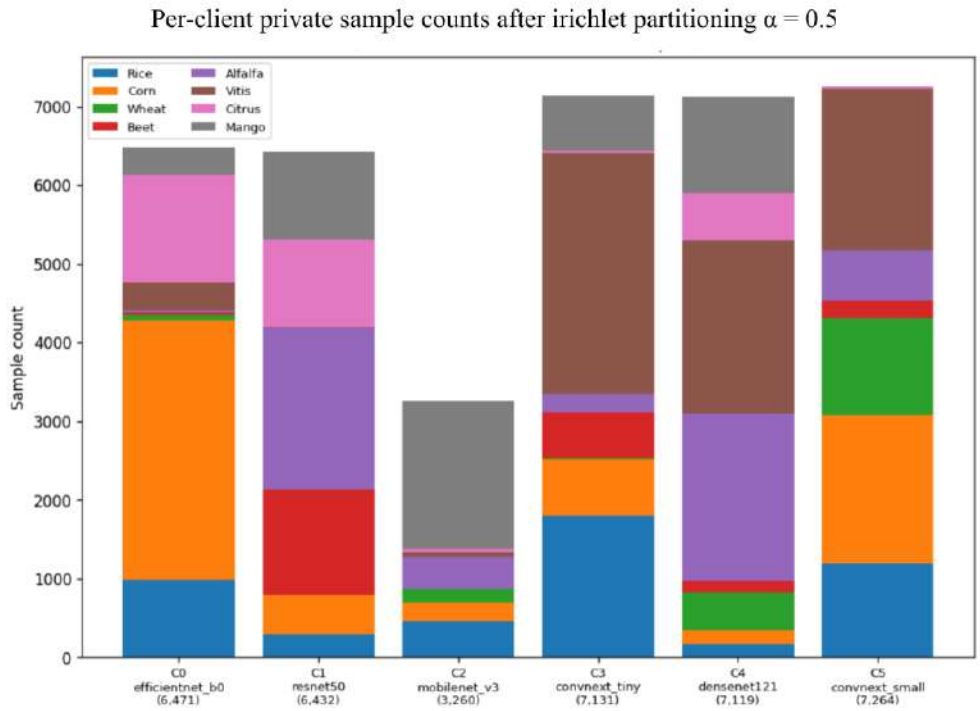


Figure 4.2: Distribution of training samples across clients after Dirichlet partitioning with $\alpha = 0.5$ at the superclass level.

To ensure reproducibility, the Dirichlet draws and the within-superclass sample shuffles are seeded with two independent random number generators initialised from the same seed (`seed = 42`). It prevents the RNG state consumed by shuffling from influencing the proportion that draws an implementation detail that would otherwise cause the client data distribution to vary with pool size and produce non-comparable splits across experimental configurations.

4.2.5 Optimiser Configuration and Training Regularisation

Using AdamW with the weight decay $\lambda = 10^{-3}$ the clients are trained. To avoid backbone feature representations from drifting aggressively during early federation rounds - which would negatively impact the projected embeddings before the shared classifier has converged - a differential learning rate schedule is applied. The learning rate for the backbone parameters is set to 5×10^{-5} (10% of the base rate), and the projection MLP and the classifier head receive 5×10^{-4}. Both sets of parameters are decayed together under a cosine annealing schedule with minimum learning rate of 10^{-5} and period equal to the total number of federation rounds. FedProx regularisation ($\mu = 0.01$) is applied exclusively to the shared classifier head, which penalises divergence of local head parameters from the global aggregated state and stabilises convergence under non-IID conditions. Gradient norms are limited to 1.0 and batches producing non-finite loss or gradient norms are discarded with a logged warning. Though these occurrences are infrequent (fewer than two per round on average) and do not significantly impact training dynamics.

4.3 Data Collection and Preparation

4.3.1 Data Collection

The primary dataset used in this research is the IP102 benchmark [2], a large-scale, publicly available collection for insect pest recognition in agricultural settings. IP102 was sourced by aggregating images from web crawling, agricultural pest monitoring databases, and field photography. It covered 102 distinct pest subclasses that are taxonomically organised into 8 crop-host superclasses such as Rice, Corn, Wheat, Beet, Alfalfa, Vitis, Citrus, and Mango. The dataset was downloaded from the official repository in its pre-partitioned form, with the training, validation, and test splits defined by the original authors. No additional images were collected or web-scraped for this work. The experimental setup exclusively uses the standard IP102 splits to maintain comparability with existing benchmarks.
The raw dataset contains 75,222 images organised in a class-folder hierarchy. Each sub-directory is identified by its integer subclass identifier (0-101) with JPEG or PNG images of variable resolution. The standard split allocates 45,095 images to training, 7,508 to validation, and 22,619 to testing, yielding an approximate ratio of 60/10/30. While some classes have few images (e.g. subclass 36 with 32 images per class) others have many images (e.g. subclass 67 with more than 2,500 images per class), which has a significant impact on the selection of a clustering based buffering strategy outlined in Section 4.3.5 and also a non-IID partitioning strategy outlined in Section 4.2.4. The per-subclass sample count distribution across all 102 classes is shown in Figure 4.3.

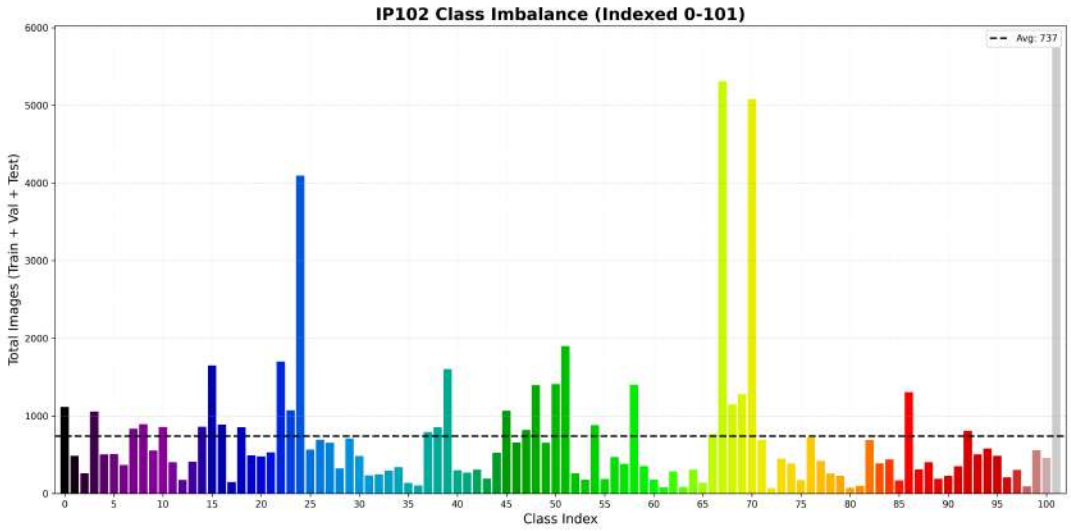


Figure 4.3: Per-subclass sample counts in the IP102 training set.

The hierarchical taxonomy is explicitly used in the data partitioning protocol described in Section 4.2.4, in which non-IID skew is induced at the superclass level instead of the subclass level. This option preserves realistic within-farm specialisation patterns where a client representing a rice-farming region holds predominantly rice-pest images. It ensures that the Dirichlet concentration parameter controls interpretable distributional skew. The proportional distribution of training samples across the eight superclasses is shown in Figure 4.4.

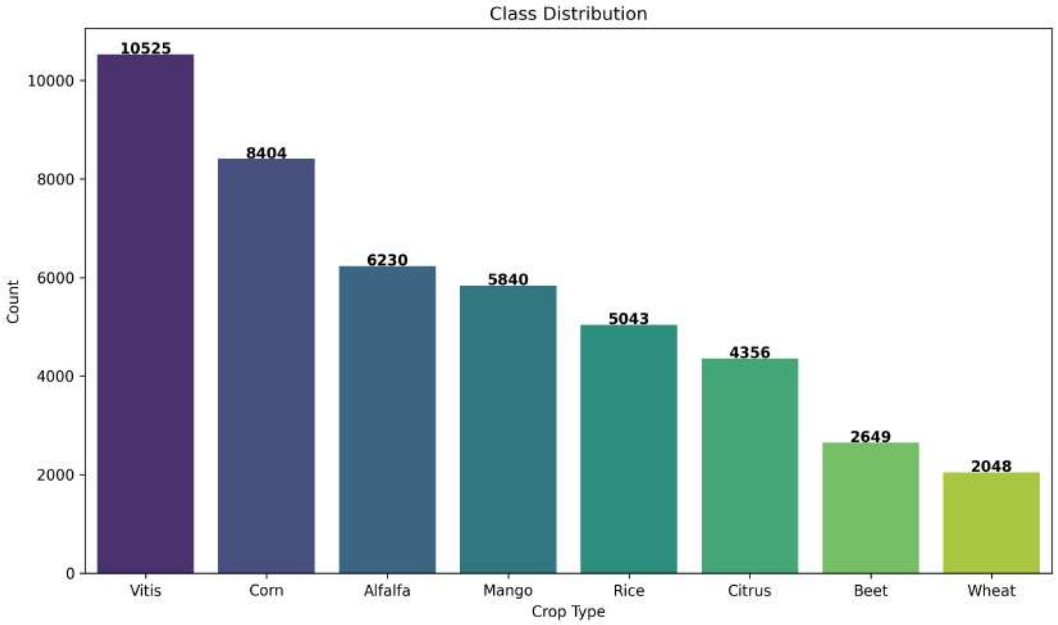


Figure 4.4: Proportion of training samples per superclass in the IP102 dataset.

4.3.2 Data Cleaning

The IP102 dataset is a curated benchmark released with a fixed split, and therefore arrives with a higher baseline quality than unprocessed web-scraped datasets. However, it was preceded by a light cleaning pass, before the data set was put into the FL pipeline.

- **Image readability validation:** The image paths are checked for being possible to open as an RGB PIL image at time of dataset construction. The files that fail to load due to corruption, unsupported formats, or encoding errors are silently ignored, and their global indices are not added in the sample list. In practice, no corrupted files were found in the standard IP102 distribution. However, the guard is retained for reproducibility across different download mirrors and decompression environments.
- **File format normalisation:** The dataset contains images with `.jpg`, `.jpeg`, and `.png` extensions. All of these are loaded uniformly via PIL and then converted to RGB colour space prior to any transform application, eliminating format-specific differences in colour channel ordering or bit depth.

- **Global index consistency:** During the construction of the dataset, each image is assigned a globally unique index number in turn, sorted by numerically oriented subdirectories. Throughout the pipeline this index is used as the third element of each sample tuple `(path, sub_label, global_idx)` to determine membership to buffers, and it is stored as a global index. Images that do not pass the readability check, do not receive a global index, and the counter increments regardless of whether a file is successfully registered. This ensures the index assignments remain consistent across independent dataset instantiations with the same directory structure.

- **Label range verification:** Subclass labels are derived directly from subdirectory names (integer strings 0-101). The taxonomy mapping (subclass to superclass) is validated at import time to ensure that all 102 subclass identifiers map to one of the 8 valid superclass identifiers, and there are no out-of-range or unmapped labels present in the dataset.

4.3.3 Data Transformation

Image transformation is applied at the `__getitem__` level within the `IP102Dataset` class, so it is not stored as pre-processed files but applied during the iteration of the dataloader. This leaves out of the picture disk I/O overhead for augmented copies and guarantees that each training epoch is different, but shows the same images.
The transformation pipelines are separated into training and evaluation modes. The same sharing of the inputs is used in all experimental setups in this paper. These are the no-buffer baseline, the homogeneous baseline and the proposed heterogeneous system. This way, no performance differences can be blamed on augmentation discrepancy.
Training transforms: The pipeline applies, Random resized cropping $[0.6, 1.0]$ and default aspect ratio perturbation, random horizontal flip, random vertical flip 20% probability, RandAugment (2 operations per image, magnitude 7), convert to 3 channel float tensor, ImageNet-standard channel normalisation ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$); random erasing 35% probability, relative area scale $[0.02, 0.20]$, and aspect ratio $[0.3, 3.3]$. Fixed photometric jitter (ColorJitter) was rejected in favour of RandAugment because it collapses several operations (colour, contrast, sharpness, and brightness) into a single hyperparameter search for the best jittering policy, rather than a search over individual jittering magnitudes. The random erasing step is performed after the normalisation step and is conducted on the tensor representation, where the pixels of a rectangular patch are set to be zero to simulate partial occlusion. The full training augmentation pipeline is illustrated in Figure 4.5.
Evaluation transforms: Validation and test images are resized to 256 pixels on the shorter edge, centre-cropped to 224×224, converted to tensor, and normalised identically. No stochastic operations are applied, ensuring deterministic evaluation.

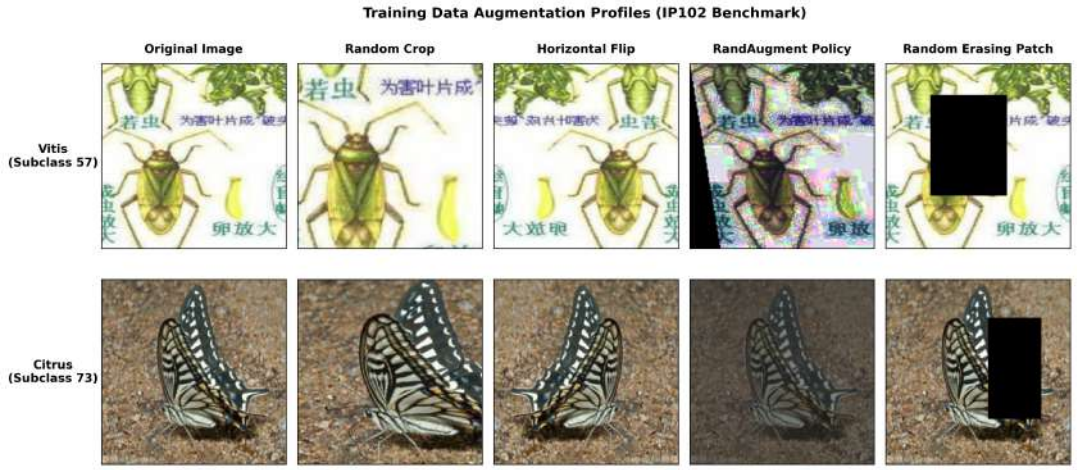


Figure 4.5: Overview of the data augmentation pipeline applied during training.

4.3.4 Data Integration

Several integration steps are required to compose the raw IP102 dataset into the structured datasets consumed by the FL pipeline.

Taxonomy integration: The mapping between the 102 subclass and the 8 superclass is specified in another taxonomy module and loaded at runtime. This mapping governs the superclass-stratified Dirichlet partition (Section 4.2.4), the buffer subclass selection criterion, and the per-class performance decomposition in the evaluation framework. Having a shared module for taxonomy instead of having multiple copies in multiple scripts ensures that the mapping is consistent throughout the partitioning, training and evaluation process.

Global index as the integration key: The global index assigned during dataset construction serves as the universal identifier that links a sample across three distinct dataset views. Such as the full training set (`IP102Dataset`), the buffer index set (`clustered_buffer_indices.pt`), and the client private subsets (`Subset` objects derived from the Dirichlet split). Global indices are used to represent buffer membership. These samples are filtered out of the Dirichlet pool using the pre-grouping index filter. During training, buffer-subclass samples are identified by matching each image's subclass label against the buffer subclass identifier set. This design avoids coupling the dataset classes to FL-specific logic, maintaining clean separation of concerns.

Split integration with the FL framework: Evaluation transforms are loaded on the validation and test splits and these splits are identical for all of the participants (server and all six clients) during the experiment. The fact that validation and test images are never included in any training partition means that there is no risk of data leakage, no matter how the training set is being split.

4.3.5 Data Reduction

The primary data reduction step in this work is the construction of the clustered buffer, which reduces the 45,095-image training set to a compact 7,418-image representative subset (16.4%) spanning 32 of the 102 pest subclasses. This reduction serves two purposes. Firstly, it reduces the volume of data the server must store

and process for generating soft-labels in each federation round. Secondly, it forces the buffer selection to be principled rather than arbitrary to ensure that the selected subclasses are those for which the backbone feature space provides the most separable representations.
The reduction process proceeds in four stages: (i) extraction of features by using a pretrained backbone applied to all training images; (ii) per-subclass centroid computation; (iii) hierarchical agglomerative clustering of centroids to 18 clusters with average linkage and a distance threshold $\tau = 0.10$; and (iv) selection of the most representative subclasses per cluster by measuring the within-cluster cohesion, up to a target buffer size. The resulting buffer inherits the class imbalance properties of the source dataset. Vitis and Corn subclasses are disproportionately represented but the clustering step ensures that the selected subclasses span the diversity of the full feature space without over-sampling the majority classes.
The secondary data reduction is the Dirichlet partition itself where each client receives only a subset of the training data (3,260 to 7,264 images depending on the client). It is a deliberate reduction designed to simulate realistic data scarcity and non-IID conditions at the edge. The end-to-end buffer construction pipeline is illustrated in Figure 4.6.

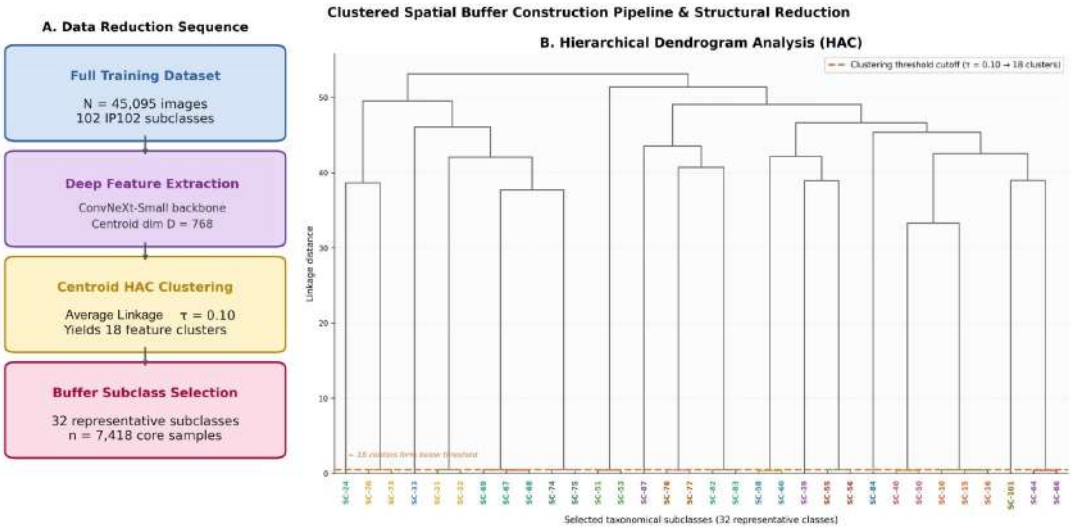


Figure 4.6: Overview of the buffer construction pipeline.

4.3.6 Data Preprocessing Workflow

The entire data preprocessing workflow, from raw IP102 files to FL-ready partitioned datasets, is summarised in Figure 4.7. The workflow proceeds in two phases: an offline preparation phase performed once before any FL training, and an online phase that operates dynamically during each federation round.

Offline phase: The training split is loaded and globally indexed. A pretrained backbone is used to extract feature vectors for all 45,095 training images, where the evaluation transforms (no augmentation) make sure that clustering is based on stable image content, not augmented views. Centroids are

computed per subclass, clustered via HAC, and 32 representative subclasses are selected to form the buffer. The buffer indices are stored and persisted to `clustered_buffer_indices.pt`. The remaining training images (37,677 after buffer exclusion) are partitioned among six clients using the superclass-level Dirichlet protocol with $\alpha = 0.5$. Client subsets are defined as index-based `Subset` objects pointing into the original `IP102Dataset`, so that training transforms are applied on-the-fly and no data is duplicated on disk.

Online phase: The server applies training transforms to the buffer images (via the standard `IP102Dataset.__getitem__` path) to generate per-class soft-label priors at the start of each FL round. Client dataloaders apply training transforms independently to each private sample per batch. The validation and test loaders have no stochasticity in the evaluation transforms.

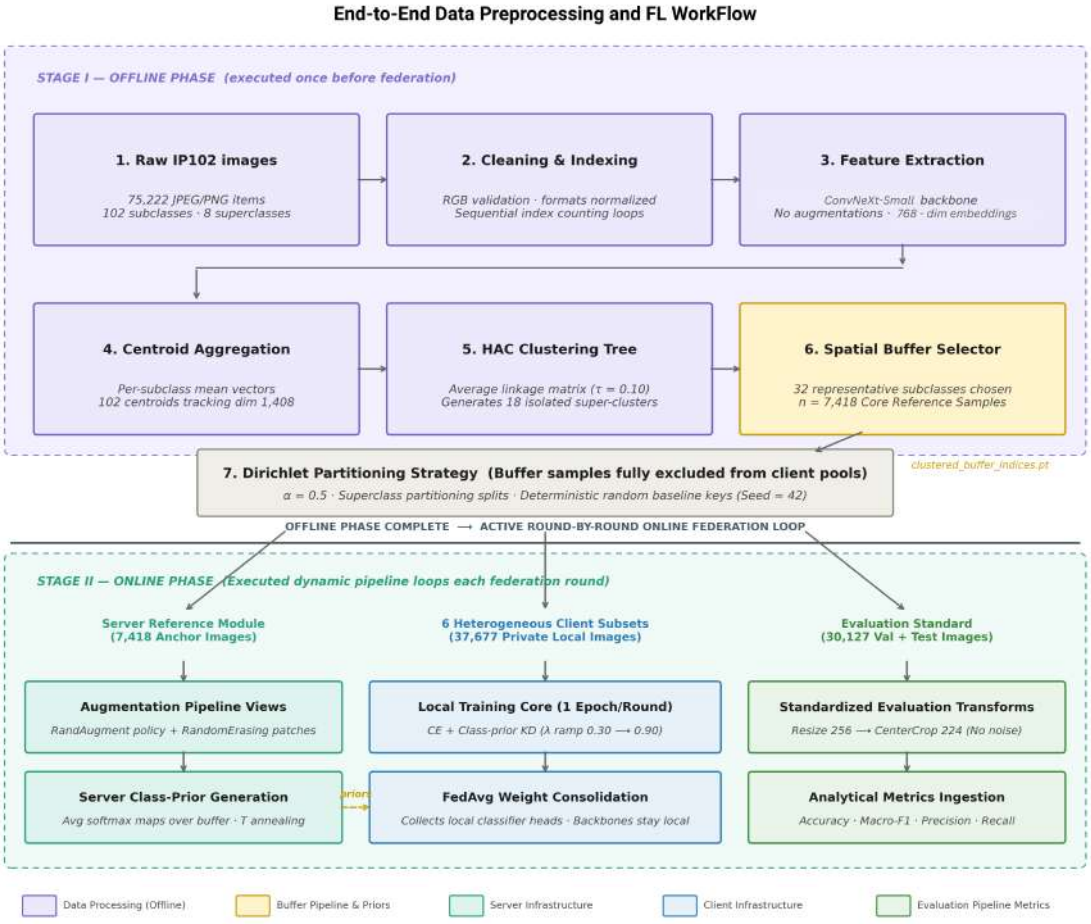


Figure 4.7: Full end-to-end workflow diagram mapping out the offline preparation and online execution stages.

4.3.7 Summary of Preprocessed Data

Table 4.3 provides a consolidated summary of the dataset statistics after all preprocessing and partitioning steps.

Table 4.3: Dataset statistics after preprocessing and partitioning. All client subsets exclude the 7,418 buffer indices but span all 102 subclasses.

Component	Samples	Sub-classes	Superclasses	Transform
Full training set	45,095	102	8	Train
Server buffer	7,418	32	8	Train
Client private pool (total)	37,677	102	8	Train
Client 0 (EfficientNet-B0)	6,471	102	8	Train
Client 1 (ResNet-50)	6,432	102	8	Train
Client 2 (MobileNetV3-L)	3,260	102	8	Train
Client 3 (ConvNeXt-Tiny)	7,131	102	8	Train
Client 4 (DenseNet-121)	7,119	102	8	Train
Client 5 (ConvNeXt-Small)	7,264	102	8	Train
Validation set	7,508	102	8	Eval
Test set	22,619	102	8	Eval

Note: Each client's private pool excludes the 7,418 buffer indices but retains samples from all 102 subclasses, including buffer subclasses (as 80% of buffer-subclass images remain in the private pool after the top 20% per subclass is allocated to the buffer).

Table 4.4 reports the per-superclass sample distribution and buffer coverage across the IP102 training set. Training sample counts are estimated proportionally from the test-set class distribution.

Table 4.4: Per-superclass sample distribution and buffer coverage in the IP102 training set. Training samples are estimated from the test-set class distribution scaled to the 45,095-sample training split. Buffer subclass counts are confirmed from the HAC clustering output.

Superclass	Crop host	Sub-classes	Training samples	Training share	Buffer sub-classes	Buffer share
Rice	*Oryza sativa*	14	≈5,041	11.2%	1	3.1%
Corn	*Zea mays*	13	≈8,388	18.6%	5	15.6%
Wheat	*Triticum* spp.	9	≈2,099	4.7%	1	3.1%
Beet	*Beta vulgaris*	8	≈2,649	5.9%	2	6.3%
Alfalfa	*Medicago sativa*	13	≈6,220	13.8%	5	15.6%
Vitis	*Vitis vinifera*	16	≈10,504	23.3%	8	25.0%
Citrus	*Citrus* spp.	16	≈3,905	8.7%	9	28.1%
Mango	*Mangifera indica*	13	≈6,289	13.9%	1	3.1%
Total		**102**	**45,095**	**100%**	**32**	**100%**

4.4 Implementation of Selected Design

4.4.1 Overview of the Federated Learning Protocol

The design chosen is a single-phase federated learning system where six heterogeneous clients communicate over 100 rounds and a central server coordinates the communications. In each round, the server sends the global classifier head to all clients, clients do a single epoch of local training, and locally updated classifier heads are sent back to the server for aggregation. The rest of the client's model - the backbone encoder, and the projection MLP - does not leave the client.
Algorithm 1 formalises the protocol.

Algorithm 1 Heterogeneous FL with Server-Anchored Class-Prior KD

Require: Server model f_{server}, buffer $\mathcal{B}$, buffer subclasses $\mathcal{S}_{\text{buf}}$, client datasets $\{D_i\}_{i=1}^{6}$, rounds $R = 100$, $\lambda_{\min} = 0.30$, $\lambda_{\max} = 0.90$, $T_{\max} = 4.0$, $T_{\min} = 2.5$

Server pretraining: Train f_{server} on $\mathcal{B}$ for 50 epochs; save best-val checkpoint
Initialise global classifier $\mathbf{W}^{(0)} \leftarrow$ pretrained server classifier head
for $t = 1, 2, \ldots, R$ **do**
    Compute $\lambda(t) \leftarrow \lambda_{\min} + (\lambda_{\max} - \lambda_{\min})\frac{t-1}{R-1}$
    Compute $T(t) \leftarrow T_{\max} - (T_{\max} - T_{\min})\frac{t-1}{R-1}$
    Generate class priors: $\mathcal{P}^{(t)} \leftarrow \left\{p_s^{(t)}\right\}_{s \in \mathcal{S}_{\text{buf}}}$ ▷ Eq. 4.6
    for each client $i = 1, \ldots, 6$ **do** ▷ in parallel
        Receive $\mathbf{W}^{(t)}$ and $\mathcal{P}^{(t)}$ from server
        $\mathbf{W}_i^{(t+1)} \leftarrow \textsc{LocalTrain}(D_i, \mathbf{W}^{(t)}, \mathcal{P}^{(t)}, \lambda(t), T(t))$ ▷ Eq. 4.10
        Send $\mathbf{W}_i^{(t+1)}$ to server
    end for
    FedAvg: $\mathbf{W}^{(t+1)} \leftarrow \sum_{i=1}^{6} \frac{|D_i|}{\sum_j |D_j|} \mathbf{W}_i^{(t+1)}$
    Evaluate ensemble on validation set; save checkpoint if best val accuracy
end for
Load best-val checkpoint; evaluate ensemble on held-out test set

The protocol differs in three ways from FedAvg. First, only the model's classifier head (26,112 parameters) aggregates, not the entire model. Second, the server randomly generates and broadcasts per-class soft-label priors at the beginning of each round, and uses per-class soft-label priors as an additional knowledge distillation signal to add to the standard cross-entropy loss on each client. Third, a progressive schedule controls the distillation weight and temperature throughout rounds as mentioned in Section 4.4.6.

4.4.2 Server Pretraining on the Clustered Buffer

The server backbone (EfficientNet-B2) is pretrained on the clustered buffer as a standalone 102-class classifier before federation begins. This step has two purposes. Firstly, it learns a server model with class-conditional representations that are specialised to the IP102 dataset's pest taxonomy, so that the soft-label priors it later broadcast are meaningful to clients. Second, the pretrained classifier head offers a rational warm start for the global aggregation state, instead of the random start

which would otherwise require more federation rounds before the shared head can produce accurate predictions.
Server pretraining adopts AdamW with learning rate 10^{-3}, weight decay 10^{-4}, cosine annealing over 50 epochs, and label smoothing 0.1. The buffer dataset is also filled with the same transforms as used for client's private data (Section 4.2.4) to ensure the feature representations used by the server are aligned with the "enhanced" image distribution seen by the clients during the federation process. Training is carried out for 50 epochs and the best checkpoint in the validation set gives a validation accuracy of 44.73% and an accuracy of 45.01% on the test set. This test accuracy is done on the server and serves as the minimum that the federated ensemble must achieve, giving a definite lower bound on the federated benefit. This server-only test accuracy constitutes the baseline that the federated ensemble is required to exceed, providing a concrete lower bound for the federation benefit. The training dynamics of the server pretraining phase are shown in Figure 4.8.

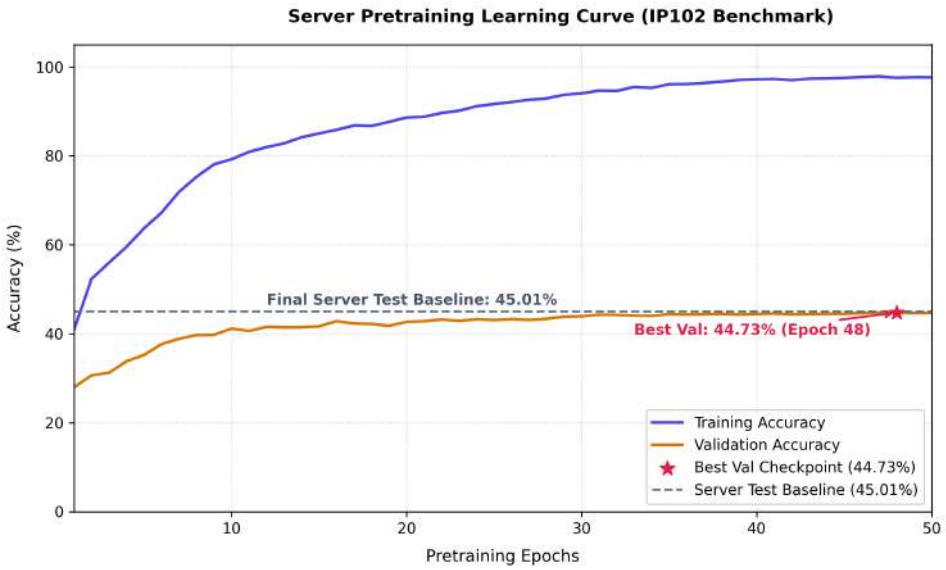


Figure 4.8: Server pretraining learning curve for EfficientNet-B2 trained on the 7,418-sample clustered buffer over 50 epochs.

Once pretrained, the best-validation checkpoint is loaded in the server model and the model is kept constant during the federation rounds, being in evaluation mode during inference (soft-label generation).

4.4.3 Server-Anchored Class-Prior Knowledge Distillation

At the start of each federation round t, the server generates a set of class-prior soft labels $\mathcal{P}^{(t)} = \{p_s^{(t)} : s \in \mathcal{S}_{\text{buf}}\}$ by performing a forward pass over all 7,418 buffer images and averaging the resulting softmax distributions per buffer subclass:

$$p_s^{(t)} = \frac{1}{|\mathcal{B}_s|} \sum_{\mathbf{x} \in \mathcal{B}_s} \sigma\left(\frac{f_{\text{server}}(\mathbf{x})}{T(t)}\right), \quad s \in \mathcal{S}_{\text{buf}} \tag{4.6}$$

where $\mathcal{B}_s$ is the set of buffer images belonging to subclass s, $f_{\text{server}}(\mathbf{x}) \in \mathbb{R}^{102}$ is the server's logit vector for image $\mathbf{x}$, $T(t)$ is the distillation temperature at round

t (annealed from 4.0 to 2.5, Section 4.4.6), and $\sigma(\cdot)$ denotes the softmax function. This gives a 32-dimensional vector, one for each buffer subclass, of dimension 102, that forms the class-prior distribution dictionary $\mathcal{P}^{(t)}$, which is sent to all clients at the initial of round t.
By performing this averaging over all buffer images in a subclass instead of doing it per-image, rather than sending 7,418 $\times$ 102 floats per round, the communication overhead is reduced to only 32×102 floats (a 231-fold lower overhead). The trade-off, that the intraclass variation of the soft labels is reduced to class mean, is partially compensated by temperature annealing schedule which is sharpening the priors as the client representations stabilize.

4.4.4 Client Local Training Objective

During each federation round, client i receives the global classifier state $\mathbf{W}^{(t)}$ and the class-prior dictionary $\mathcal{P}^{(t)}$. It then trains on its private dataset D_i for one epoch. If the subclass labels are given for images in a mini-batch, the loss is calculated both in the buffer-subclass and non-buffer-subclass mini-batches.
Let $\mathcal{B}_{\text{batch}}$ and $\mathcal{N}_{\text{batch}}$ be the sets of buffer-subclass and non-buffer-subclass samples in the current batch, respectively, with cardinalities n_b and n_n respectively, and $n = n_b + n_n$.
Cross-entropy with non-buffer amplification:

$$\mathcal{L}_{\text{CE}} = \frac{n_b \cdot \mathcal{L}_{\text{CE}}^{\mathcal{B}} + n_n \cdot \gamma \cdot \mathcal{L}_{\text{CE}}^{\mathcal{N}}}{n} \tag{4.7}$$

where $\mathcal{L}_{\text{CE}}^{\mathcal{B}}$ and $\mathcal{L}_{\text{CE}}^{\mathcal{N}}$ are the cross-entropy losses (with label smoothing 0.1) computed on buffer-subclass and non-buffer-subclass samples respectively, and $\gamma = 1.5$ is the non-buffer amplification factor. The weighting gives all 70 non-buffer classes the same number of "gradient feet" to play with as the 32 buffer classes that get both CE and KD supervision.
Class-prior knowledge distillation: The client applies KL-divergence distillation for each buffer-subclass sample $(\mathbf{x}_j, s_j) \in \mathcal{B}_{\text{batch}}$, to the respective class-prior $p_{s_j}^{(t)}$:

$$\mathcal{L}_{\text{KD}} = T(t)^2 \cdot \frac{1}{n_b} \sum_{j \in \mathcal{B}_{\text{batch}}} \text{KL}\left(\sigma\left(\frac{\mathbf{y}_j}{T(t)}\right) \,\middle\|\, \sigma\left(\frac{p_{s_j}^{(t)}}{T(t)}\right) \right) \tag{4.8}$$

where $\mathbf{y}_j$ is the student (client) logit vector for image $\mathbf{x}_j$ and the $T(t)^2$ factor preserves gradient magnitude under temperature scaling.
FedProx regularisation: The proximal term is added to prevent the local classifier from drifting too far from the global state in the single local epoch:

$$\mathcal{L}_{\text{prox}} = \frac{\mu}{2} \left\| \mathbf{W}_{\text{local}} - \mathbf{W}^{(t)} \right\|_F^2 \tag{4.9}$$

with $\mu = 0.01$ applied exclusively to the shared classifier head parameters.
Total local objective:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \lambda(t) \cdot \mathcal{L}_{\text{KD}} + \mathcal{L}_{\text{prox}} \tag{4.10}$$

The weight of the distillation $\lambda(t)$ is the weight that is dependent on the round as described in Section 4.4.6.

If a batch consists of no buffer-subclass samples, e.g. clients have highly skewed non-IID data, with the superclasses for the buffer data being non-buffer, then the KD term is not included, and only $\mathcal{L}_{\text{CE}}$ and $\mathcal{L}_{\text{prox}}$ contribute to the update. This is handled naturally by the buffer-subclass mask, which produces an empty set for $\mathcal{B}_{\text{batch}}$ in such cases.

4.4.5 Federated Aggregation and Global Classifier Update

When the local training epoch for each client finishes, it sends its new state of the classifiers $\mathbf{W}_i^{(t+1)}$ to the server. For the new global classifier, the server calculates the FedAvg based on the number of samples.

$$\mathbf{W}^{(t+1)} = \sum_{i=1}^{6} \frac{|D_i|}{\sum_{j=1}^{6} |D_j|} \mathbf{W}_i^{(t+1)} \tag{4.11}$$

where $|D_i|$ denotes the number of private training samples of client i. The weights have a distribution from 8.6% (Client 2, MobileNetV3-Large, 3,260 samples) to 19.3% (Client 5, ConvNeXt-Small, 7,264 samples) and is therefore not evenly distributed because of the Dirichlet split. The results of the aggregation function $\mathbf{W}^{(t+1)}$ are then broadcast to all clients at the beginning of round $t+1$, overwriting the classifier state of each client before the next training epoch.
The server backbone and global classification head are two model parts. The shared head is the head that all clients use, it isn't part of the server backbone head. Only the parameters of the server backbone's classifier are updated after pretraining, for generating soft labels. This architectural isolation ensures that there has been no contamination between aggregated client knowledge and the distillation source on the server.

4.4.6 Progressive Knowledge Transfer Schedule

A fixed distillation weight and temperature applied uniformly across all 100 rounds would impose the same KD pressure in round 1 - when client backbone features are still adapting from ImageNet initialisation to the IP102 distribution - as in round 90, when the features are well-specialised. To address this, both the distillation weight and temperature are varied linearly across rounds.
Distillation weight ramp:

$$\lambda(t) = \lambda_{\min} + (\lambda_{\max} - \lambda_{\min}) \cdot \frac{t-1}{R-1} \tag{4.12}$$

with $\lambda_{\min} = 0.30$, $\lambda_{\max} = 0.90$, and $R = 100$. At round 1, $\lambda(1) = 0.30$; at round 100, $\lambda(100) = 0.90$. The low initial weight allows client backbones to fit the private data distribution through CE supervision before the KD signal begins to dominate. As training progresses and backbone features stabilize, the ramp increases the relative contribution of the teacher's knowledge.
Temperature annealing:

$$T(t) = T_{\max} - (T_{\max} - T_{\min}) \cdot \frac{t-1}{R-1} \tag{4.13}$$

with $T_{\max} = 4.0$ and $T_{\min} = 2.5$. High temperatures in early rounds produce soft probability distributions that convey relational similarity among classes without overconfidently prescribing specific predictions. As rounds progress and the client classifiers become more discriminative, lower temperatures yield sharper priors that provide more precise directional guidance. Both schedules are plotted over the 100 federation rounds in Figure 4.9.

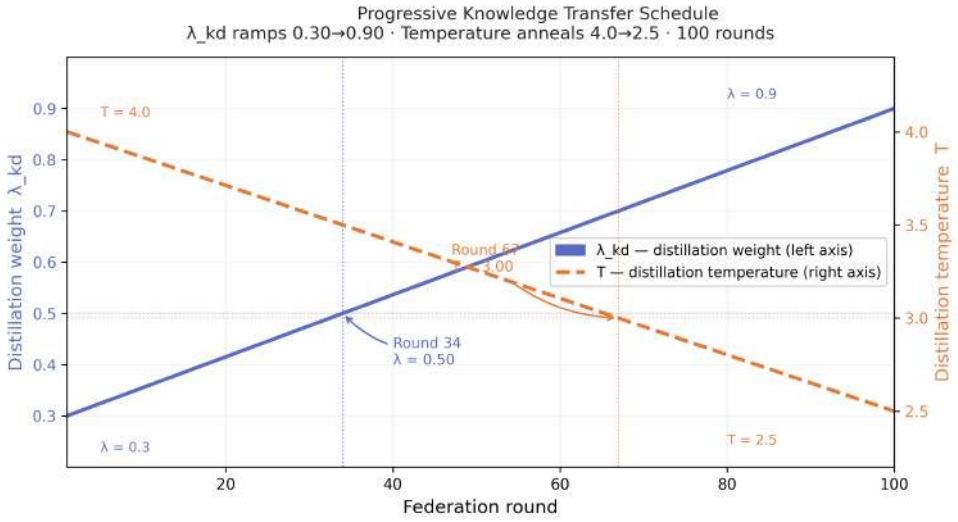


Figure 4.9: Progressive knowledge transfer schedule over 100 federation rounds.

Learning rate schedule. Each client's optimiser uses a cosine annealing schedule with period equal to the total number of rounds (100) and minimum learning rate 10^{-5}, applied to both backbone and head parameter groups. The backbone group operates at 10% of the head learning rate (5×10^{-5} vs 5×10^{-4}) to prevent large feature drift between rounds.

4.4.7 Privacy Preservation through Architectural Incommensurability

An important property of the proposed implementation is that privacy is guaranteed by architecture itself and not by post-hoc noise injection. The mechanism relies on the observation that a property inference attack against an FL system requires the adversary (the server) to compare gradient update vectors across clients to learn statistical characteristics of their private data. This comparison is only meaningful when the parameter vectors are in a common space - i.e., if all clients share the same architecture and the same parameter indexing.

In the proposed system, each client uses a distinct backbone architecture with a distinct number of parameters, different layer widths, and different weight tensor shapes (e.g., ResNet-50 has 25.6M backbone parameters vs EfficientNet-B0 at 5.3M). The backbone updates never leave the client. The only shared parameters - the classifier head ($\mathbf{W} \in \mathbb{R}^{102\times256}$, 26,112 parameters) - are architecturally identical across clients by design, since all clients project to the same 256-dimensional embedding space before classification.

An empirical quantisation of the residual leakage through the shared head is performed, by carrying out a property inference attack. The homogeneous baseline of

all six clients share an identical ResNet-50 architecture and full-model gradient updates occupy a common, directly comparable parameter space. It produces attack accuracy of 56.7-61.7%, which is 4.5-4.9 times above the 12.5% random baseline. The heterogeneous design reduces this to 0%, since the backbone updates (which carry the majority of the private data signal) are not shared, and the shared head carries only a highly compressed representation, the attack has no exploitable gradient structure to learn from. This is the complete elimination of the main attack surface, without the introduction of accuracy-reducing Gaussian noise and without the need for secure multi-party computation. Table 4.5 summarises the attack accuracy across all three configurations.

Table 4.5: Privacy attack accuracy comparison across FL configurations. Attack target: dominant crop-host superclass inference from gradient updates (8-class problem, random baseline 12.5%).

Setup	**Parameters shared**	**Attack acc.**	**Random baseline**	**Relative to random**
Homo FL, full-model FedAvg	Full model (≈25.6M)	56.7-61.7%	12.5%	4.5-4.9×
Hetero FL, full-model (infeasible)	N/A (incommensurable dims)	N/A	12.5%	N/A
Hetero FL, classifier only (proposed)	Head only (26,112 params)	**0.0%**	12.5%	**0**×

4.4.8 Homogeneous FL Baseline Configuration

A homogeneous FL baseline is established alongside the proposed heterogeneous system to serve two distinct purposes. To quantify the privacy cost incurred when architectural uniformity is extended across clients, and to provide a performance reference point that isolates the effect of heterogeneity from the effect of the buffer and KD mechanisms.

In the homogeneous configuration, all six clients are assigned identical ResNet-50, efficientNet-B0, efficientNet-B2 backbone networks. Unlike the proposed system, where only the classifier head participates in aggregation, the homogeneous baseline performs full-model FedAvg - the complete parameter vector of each client, including backbone, projection MLP, and classifier head, is averaged at the server and broadcast back to clients each round. No shared buffer is employed, and no knowledge distillation signal is broadcast from the server. Clients train only on their private data using the standard cross-entropy objective with FedProx regularisation ($\mu = 0.01$).

For the homogeneous baseline the data partition is generated using the identical superclass-level Dirichlet protocol ($\alpha = 0.5$, seed = 42). It is applied to the full 45,095-image training set without excluding any buffer indices, since no buffer exists in this configuration. Therefore, all six clients train on disjoint subsets of the complete training corpus. Other training hyperparameters such as learning rate, weight decay, cosine annealing schedule, dropout, label smoothing, and augmentation pipeline are the same to those used in the heterogeneous configurations. It is

to ensure that any observed differences in classification accuracy or privacy vulnerability are attributable to architectural choice instead of training regime discrepancy. The critical distinguishing feature of this configuration is that full-model gradient updates occupy a common, directly comparable parameter space. Because all six clients apply gradients to structurally similar weight tensors, a server-side adversary can perform gradient similarity metrics across clients, apply regression-based inference, or train a meta-classifier over gradient update vectors to recover statistical properties of individual clients' private data. In particular, the attack targets the multi-crop portfolio problem by allowing the server to identify which of the 8 crop-host superclasses exist within the client's private training data from a single gradient update. A uniformly random predictor on this task achieves 12.5% accuracy, since there are 8 equally likely superclasses.

As reported, a property inference attack against the homogeneous baseline achieves 56.7-61.7% accuracy which is 4.5 to 4.9 times higher than the random baseline. It confirms that full-model gradient sharing in a homogeneous architecture constitutes a serious and exploitable privacy vulnerability. The overall classification accuracy of the proposed heterogeneous design is also compatible with that of the original, and achieving the same attack accuracy to 0% means that the architectural incommensurability does not compromise classification utility.

Chapter 5

Result Analysis

5.1 Performance Evaluation

This section establishes the evaluation framework used to assess all experimental configurations reported in this chapter. It defines the metrics used to measure classification performance, privacy exposure, and generalisation, and describes the testing methodology applied uniformly across every configuration.

5.1.1 Evaluation Criteria

Classification Accuracy

The primary classification metric is **overall test accuracy**, defined as the fraction of test images for which the ensemble's predicted subclass label matches the ground-truth label:

$$\text{Accuracy} = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{(\mathbf{x},y)\in\mathcal{D}_{\text{test}}} \mathbf{1}[\hat{y}(\mathbf{x}) = y] \tag{5.1}$$

where $|\mathcal{D}_{\text{test}}| = 22{,}619$. **Macro-averaged F1 score** is the secondary metric: unlike overall accuracy, it penalises configurations that achieve high accuracy by concentrating performance on large majority classes at the expense of smaller ones - an important property given the severe class imbalance of IP102 (subclass sizes range from 32 to over 2,500 training images). Macro-averaged precision and recall are also reported to characterise the direction of classification errors.

Per-Class Accuracy Decomposition

A critical evaluation criterion specific to this work is the **per-class accuracy decomposition**, which partitions the 102 test subclasses into two non-overlapping groups:

- **Buffer subclasses** ($|\mathcal{S}_{\text{buf}}| = 32$): subclasses for which the server holds buffer images and generates class-prior KD priors each round.
- **Non-buffer subclasses** ($|\mathcal{S}_{\text{non}}| = 70$): subclasses entirely absent from the server buffer; their performance depends solely on client private data.

Group-level accuracy is computed separately for each partition:

$$\text{Acc}_{\text{buf}} = \frac{\sum_{s \in \mathcal{S}_{\text{buf}}} \text{correct}(s)}{\sum_{s \in \mathcal{S}_{\text{buf}}} \text{total}(s)}, \qquad \text{Acc}_{\text{non}} = \frac{\sum_{s \in \mathcal{S}_{\text{non}}} \text{correct}(s)}{\sum_{s \in \mathcal{S}_{\text{non}}} \text{total}(s)} \tag{5.2}$$

This decomposition is the primary evidence for or against *cross-class coordination*: a system that improves Acc_{non} demonstrates that its shared knowledge mechanism benefits classes it has never directly supervised. A system that improves only Acc_{buf} is specialising, not coordinating.

Privacy Exposure

Privacy is evaluated through a **property inference attack accuracy**, which measures how reliably a server-side adversary can infer the dominant crop-host superclass of a client's private data from its gradient update vector. With 8 equally likely superclasses, a uniformly random predictor achieves 12.5% accuracy. Attack accuracy above this baseline quantifies the severity of gradient leakage.

Efficiency and Reliability

Computational efficiency is characterised by the train-validation accuracy gap at the end of federation, which serves as a proxy for generalisation ceiling and overfitting severity under non-IID conditions. **Communication efficiency** of the proposed system transmits $32 \times 102 = 3{,}264$ floats per round as class priors, compared to full-model state dictionaries of up to 25.6 M parameters. **Measurement reliability** is supported by the scale of the test set: at 22,619 images, a 1 pp difference corresponds to approximately 226 correctly classified images. Comparisons of 2 pp or more are reliable at this scale; comparisons within 1-2 pp are framed as indicative throughout the chapter.

5.1.2 Testing Methods

Ensemble Inference

All heterogeneous configurations produce test predictions through an ensemble of six client models. For a given test image $\mathbf{x}$, the six softmax probability vectors are averaged element-wise:

$$\hat{y}(\mathbf{x}) = \arg\max_{c} \frac{1}{6} \sum_{i=1}^{6} \sigma(f_i(\mathbf{x}))_c \tag{5.3}$$

All models are placed in evaluation mode with all dropout layers deactivated, producing deterministic predictions.

Best-Checkpoint Selection

The model checkpoint achieving the highest validation accuracy across all federation rounds is saved and used for final test evaluation. This prevents reporting either inflated numbers from late-round drift or deflated numbers from runs that peak early.

Property Inference Attack Evaluation

The property inference attack is implemented as a cosine-similarity-based inference method. At each round t, the cosine similarity between a client's classifier head gradient update $\Delta \mathbf{W}_i^{(t)}$ and a set of eight superclass reference gradients $\{\mathbf{g}_k\}_{k=1}^{8}$ is computed:

$$\hat{k}_i^{(t)} = \arg\max_k \frac{\Delta \mathbf{W}_i^{(t)} \cdot \mathbf{g}_k}{\|\Delta \mathbf{W}_i^{(t)}\| \cdot \|\mathbf{g}_k\|} \tag{5.4}$$

Attack accuracy is averaged over all clients and all rounds, with the mean over the last 10 rounds reported as the steady-state vulnerability metric.

5.2 Analysis of Design Solutions

Five experimental configurations are evaluated in this work. The core contribution is the design of a heterogeneous federated learning system; the homogeneous federated learning configuration was implemented specifically to provide an empirical basis for the privacy motivation, demonstrating what gradient leakage looks like when architectural uniformity is present. Table 5.1 provides the full numerical overview.

Table 5.1: Numerical comparison of all experimental configurations on the IP102 test set.

Configuration	Test acc.	F1	Buf. acc.	Non-buf. acc.	Attack acc.	Rounds	
Server baseline	45.01%	0.337	82.20%	0.00%	-	50 ep.	
Homogeneous FL (EfficientNet-B0)	64.01%	0.597	-	-	60.0%	50	
Homogeneous FL (EfficientNet-B2)	64.90%	0.606	-	-	56.7%	50	
Homogeneous FL (ResNet-50)	65.75%	0.612	-	-	61.7%	50	
Heterogeneous FL, no buffer	59.60%	0.533	78.15%	33.14%	0.0%	80	
Heterogeneous FL, complex training	57.47%	0.501	-	-	0.0%	80	
Heterogeneous FL, buffer at server	**63.76%**	**0.611**	**80.02%**	**45.71%**	**0.0%**	20.7 pp	100

5.2.1 Server Pretraining Baseline

Description: The EfficientNet-B2 server backbone is trained for 50 epochs as a standalone 102-class classifier on the 7,418-sample clustered buffer alone. This configuration is not a design candidate for deployment; it establishes the lower bound for the federated benefit.

Accuracy and performance: The server achieves 45.01% test accuracy (F1 = 0.337), with 82.20% buffer-class accuracy and exactly 0.00% non-buffer-class accuracy. Training accuracy reaches 97.7% by epoch 50, confirming over-fitting to the buffer corpus. The server pretraining learning curve is shown in Figure 4.8 (Chapter 4).

Efficiency and cost: Training completes in approximately 11 minutes on a single GPU. No communication overhead is incurred.

- **Strengths:** Highest buffer-class accuracy of all configurations (82.20%); provides a clean lower bound for measuring federated benefit.
- **Weaknesses:** Zero non-buffer generalisation; server has no idea about the non-buffer classes.

5.2.2 Homogeneous Federated Learning

Description: Three homogeneous federated learning runs are conducted with EfficientNet-B0, EfficientNet-B2, and ResNet-50, where all six clients share an identical backbone and perform full-model FedAvg each round. These runs are included not as design alternatives but to provide empirical evidence for the privacy question motivating the heterogeneous architecture: when clients share the same parameter space, how much information about their private data does the server learn?
Accuracy and performance: The three variants achieve 64.01%, 64.90%, and 65.75% test accuracy. ResNet-50 achieves the highest (F1 = 0.612). The train-validation gap is modest at 6.0-7.2 pp, substantially smaller than the heterogeneous configurations, because full-model FedAvg provides a strong shared feature representation that naturally constrains client drift. The training accuracy and validation accuracy curves for all three backbone variants are shown in Figure 5.1.

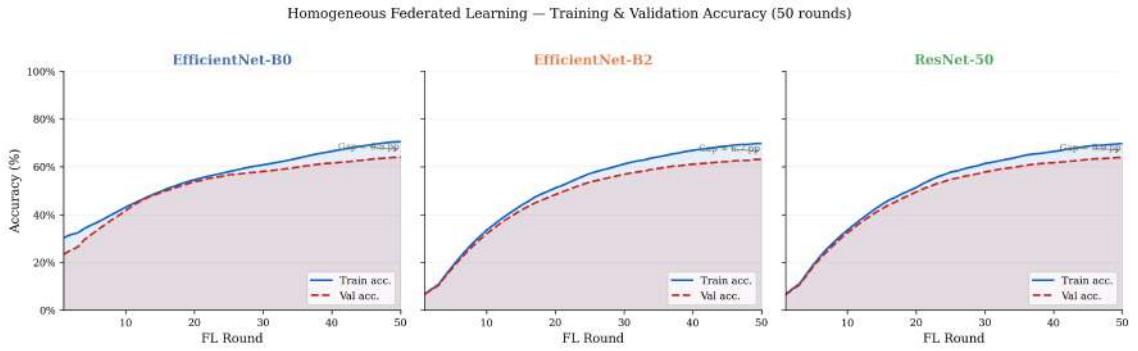


Figure 5.1: Training and validation accuracy curves for the three homogeneous federated learning configurations over 50 rounds.

Each panel shows that all three homogeneous variants converge rapidly with a consistently small train-validation gap, in contrast to the heterogeneous configurations discussed in Section 5.2. The tight gap confirms that full-model gradient sharing effectively aligns client representations under the non-IID Dirichlet partition. The corresponding cross-entropy loss curves are shown in Figure 5.2.
The loss curves confirm smooth convergence in all three configurations, with training and validation loss tracking closely throughout the 50-round budget - further evidence that full-model parameter sharing resolves the non-IID distribution mismatch at the cost of gradient privacy.
Privacy exposure: Property inference attack accuracy is 56.7-61.7%, between 4.5× and 4.9× the 12.5% random baseline. This confirms that full-model gradient sharing in a common parameter space is a severe and practically exploitable privacy vulnerability. The accuracy difference between the best homogeneous configuration (65.75%) and the proposed system (63.76%) is 1.99 pp - a gap that must be weighed against the complete elimination of a 56.7-61.7% attack surface.

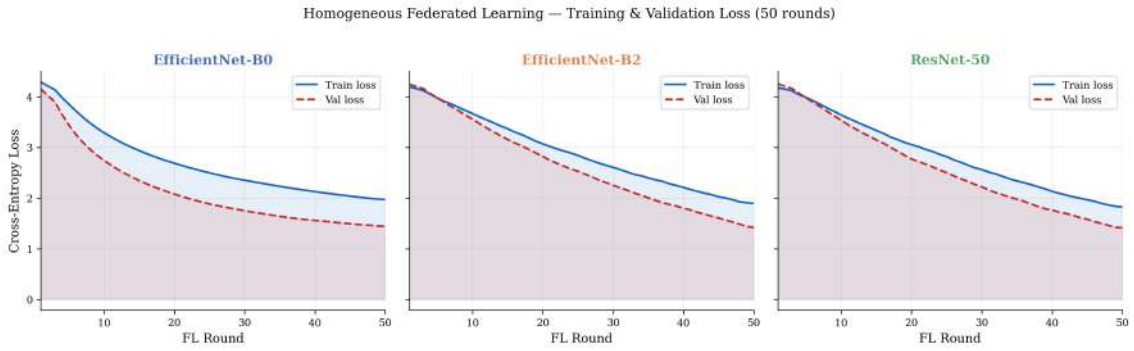


Figure 5.2: Training and validation loss curves for the three homogeneous federated learning configurations over 50 rounds.

Efficiency and cost: Full-model FedAvg aggregates up to 25.6 M parameters per client per round (ResNet-50) - approximately 980× the cost of classifier-head-only aggregation.
Feasibility: Requires all clients to deploy identical backbone architectures, mandating hardware uniformity across edge nodes - impractical in real agricultural deployments.

- **Strengths:** Highest raw test accuracy (65.75%); smallest train-val gap; fastest convergence.
- **Weaknesses:** Critical privacy failure (56.7-61.7% attack accuracy); requires hardware uniformity; high communication cost; structurally incompatible with diverse edge deployments.

5.2.3 Heterogeneous Federated Learning without Buffer

Description. Six clients with heterogeneous backbone architectures train on their private data (45,095 images total), with only the shared classifier head (26,112 parameters) aggregated each round. No buffer, no knowledge distillation, and no server pretraining is used. This is the heterogeneous privacy-safe baseline.
Accuracy and performance. The configuration achieves 59.60% test accuracy (F1 = 0.533), a 14.59 pp improvement over the server-alone baseline. Buffer-class accuracy is 78.15% and non-buffer-class accuracy is 33.14%. The 18.7 pp train-validation gap at round 80 reflects the structural mismatch between the Dirichlet-skewed training distributions and the balanced global evaluation set. The training curve for this configuration alongside the proposed system is shown in Figure 5.3.
Privacy exposure. 0% attack accuracy, achieved structurally through backbone dimensional mismatch.
Efficiency and cost. Communication overhead limited to the 26,112-parameter classifier head - 980× below full-model ResNet-50 sharing.

- **Strengths:** Structural privacy guarantee; minimal communication overhead; simplest heterogeneous configuration.
- **Weaknesses:** Low non-buffer accuracy (33.14%); large persistent train-val gap; no shared knowledge anchor.

5.2.4 Heterogeneous Federated Learning with Server-Retained Buffer

Description: The proposed system retains the 7,418-sample clustered buffer exclusively at the server, generates per-class soft-label priors broadcast to clients each round, and pretrains on the buffer for 50 epochs before federation. The distillation weight ramps from $\lambda = 0.30$ to 0.90 and temperature anneals from $T = 4.0$ to 2.5 over 100 rounds. A $\gamma = 1.5$ cross-entropy amplification prevents the classifier from specialising toward the 32 KD-supervised subclasses at the expense of the other 70. This is the only configuration that satisfies all four design objectives in Section 4.1.3.
Accuracy and performance: The proposed system achieves 63.76% test accuracy and a macro-F1 of 0.611 - the best F1 among all heterogeneous configurations. The best validation checkpoint is reached at round 91 (63.80%). Buffer-class accuracy is 80.02% and non-buffer-class accuracy is 45.71%. The key finding is the asymmetry: the non-buffer improvement over the no-buffer baseline is 12.57 pp (33.14% $\rightarrow$ 45.71%), which is 6.7$\times$ larger than the buffer-class improvement (1.87 pp). This demonstrates cross-class coordination rather than buffer-class specialisation.
The training accuracy and validation accuracy curves for the no-buffer baseline and the proposed system are compared directly in Figure 5.3.

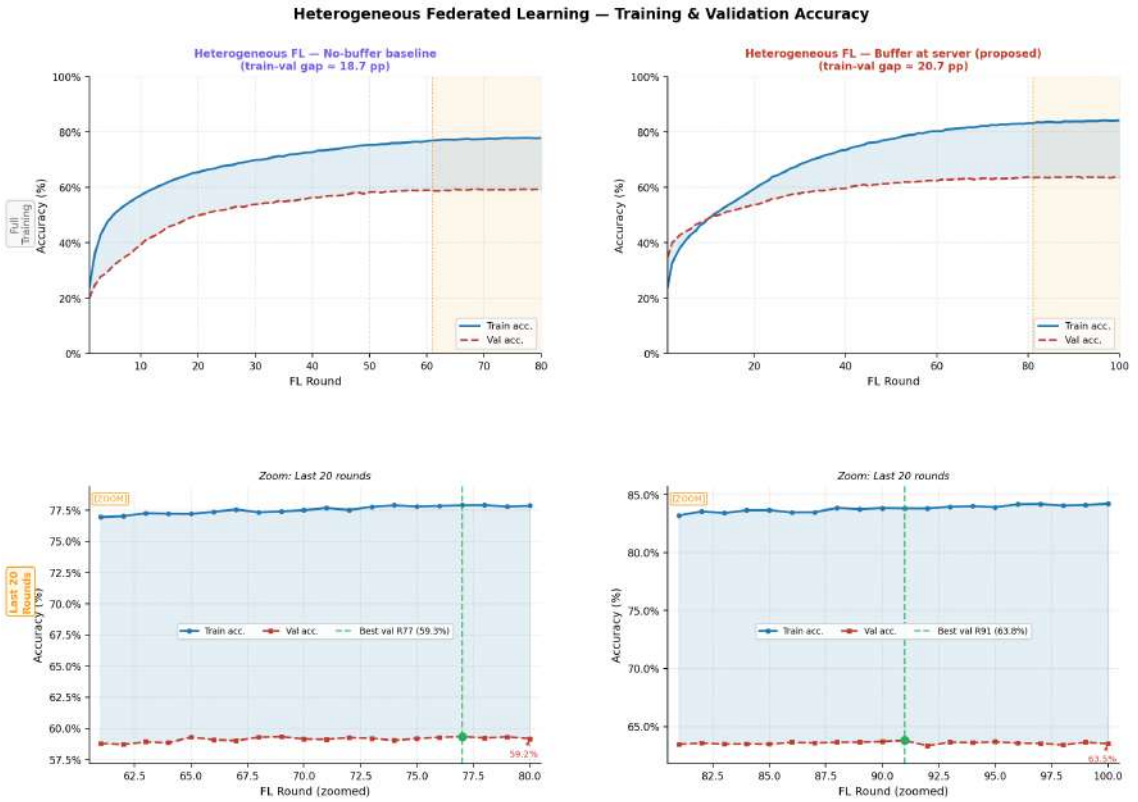


Figure 5.3: Training and validation accuracy curves for the no-buffer baseline and the proposed buffer-at-server system.

The left panel shows the 18.7 pp positive gap of the standard no-buffer run (rounds 1-80). The right panel shows the proposed buffer-at-server system over 100 rounds: the gap begins slightly negative in the first few rounds - the pretrained classifier head produces accurate validation predictions before private-data features have sta-

bilised - converges near zero around round 10, then widens progressively as the λ_{kd} ramp increases KD supervision pressure. The widening is therefore not evidence of increasing overfitting but of the growing contribution from the class-prior signal. The buffer-class and non-buffer-class accuracy comparison across the heterogeneous configurations is shown in Figure 5.4.

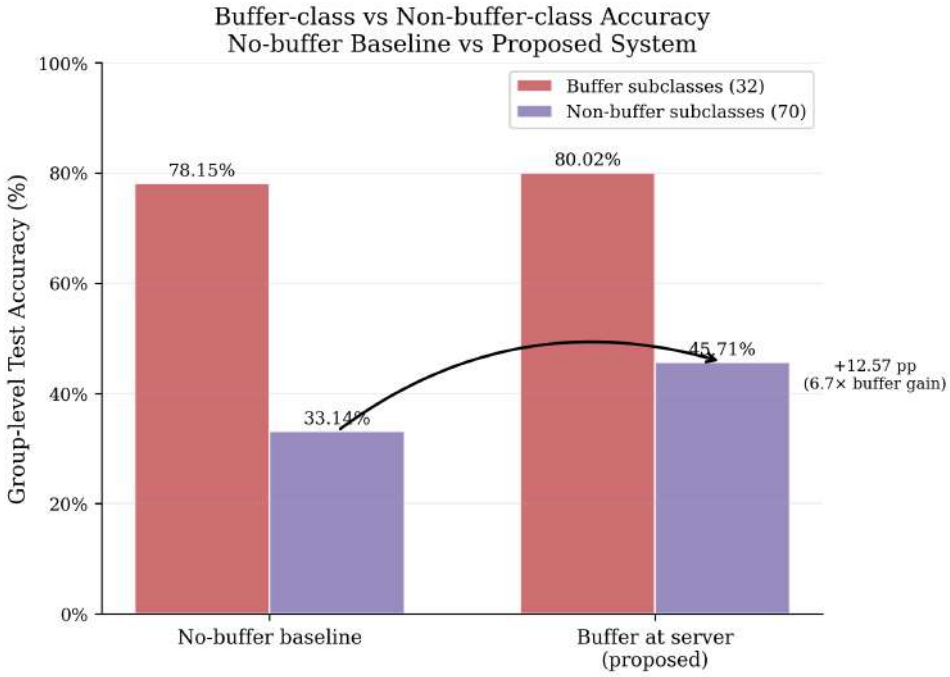


Figure 5.4: Buffer-class and non-buffer-class test accuracy for the no-buffer baseline and the proposed buffer-at-server system.

The proposed system is the only configuration that improves both accuracy groups relative to the no-buffer baseline. The non-buffer improvement is 6.7× larger than the buffer improvement, confirming that the knowledge distillation mechanism coordinates across the full taxonomy rather than specialising toward the classes it directly supervises.

Privacy exposure: 0% attack accuracy. The addition of the server buffer and class-prior broadcast introduces no new leakage: the KD priors are 32 averaged probability distributions containing no client-specific information.

Efficiency and cost: The class-prior broadcast adds $32 \times 102 = 3{,}264$ floats per round - negligible relative to the 26,112-parameter classifier head. The server's buffer forward pass takes approximately 2.1 seconds per round ($< 3\%$ of total round time).

- **Strengths:** Best F1 among heterogeneous configurations (0.611); 45.71% non-buffer accuracy; 0% privacy attack; 6.7× non-buffer/buffer improvement ratio; +18.75 pp lift over server-alone baseline.
- **Weaknesses:** 1.99 pp below the best homogeneous configuration; 20.7 pp train-val gap; 25 non-buffer subclasses remain at 0% due to data sparsity; requires server to hold and process the buffer each round.

5.3 Final Design Adjustments

The performance evaluation in Section 5.2 reveals three categories of design issues: a structural issue with buffer placement, a scheduling issue with the knowledge distillation parameters, and a generalisation gap that resisted regularisation-based correction. This section documents the adjustment made in response to each.

5.3.1 Adjustment 1: Relocating the Buffer from Clients to Server

Observation: The initial design placed buffer images at both the server and each client. Per-class evaluation showed this improved buffer-class accuracy by approximately 10 pp while simultaneously degrading non-buffer-class accuracy by approximately 5 pp. The root cause was gradient crowding - at peak distillation weight ($\lambda = 1.5$), buffer-subclass samples received approximately 2.9× the gradient magnitude of non-buffer samples, making the shared classifier head systematically overrepresent buffer-class geometry in every aggregated update.
Adjustment: The buffer was migrated exclusively to the server. Clients no longer hold buffer images. Instead, the server computes one averaged soft-label vector per buffer subclass and broadcasts 32 such vectors per round, eliminating the within-batch gradient imbalance at clients.
Effect: Non-buffer-class accuracy exceeded the no-buffer baseline for the first time. Under a fixed distillation weight of $\lambda = 0.70$, non-buffer accuracy reached 40.61% - a 7.47 pp improvement over the 33.14% baseline.

5.3.2 Adjustment 2: Progressive Distillation Schedule

Observation: With a fixed $\lambda = 0.70$, client backbone features adapt from ImageNet initialisation to IP102 in early rounds, during which strong KD pressure is counterproductive. A fixed temperature of $T = 4.0$ throughout training also provides imprecise soft-label priors in late rounds when clients can benefit from sharper directional signals.
Adjustment: The distillation weight was changed to a linear ramp from $\lambda = 0.30$ to 0.90 over $R = 100$ rounds, and the temperature was annealed from $T = 4.0$ to 2.5. The round budget was extended from 80 to 100 to match the slower early convergence of the low-λ start.
Effect: Non-buffer-class accuracy improved from 40.61% to 45.71% - a further gain of 5.10 pp. The best validation checkpoint shifted to round 91, confirming the extended budget is needed.

5.3.3 Adjustment 3: Non-Buffer Cross-Entropy Amplification

Observation: At peak $\lambda = 0.90$, buffer-subclass samples still receive 1.9× the loss contribution of non-buffer samples, creating a persistent gradient imbalance in favour of the 32 KD-supervised subclasses.
Adjustment: A non-buffer CE amplification factor $\gamma = 1.5$ was introduced, scaling the cross-entropy loss of non-buffer samples before the batch-level weighted average,

reducing the effective loss ratio from 1.9:1.0 to approximately 1.9:1.5.
Effect: Ablation without the amplification factor confirmed a 2-3 pp reduction in non-buffer accuracy when $\gamma = 1.0$, validating the adjustment as necessary for balanced cross-class performance.

5.3.4 Adjustment 4: Investigation of the Train-Validation Gap

Observation: The heterogeneous configurations produce 18.7-20.7 pp train-validation gaps, raising the question of whether further regularisation would close the gap while preserving test accuracy.
Adjustment attempted: Label smoothing was increased from 0.10 to 0.15, and MixUp ($\alpha = 0.2$) and CutMix (probability 0.5) were added.
Conclusion: The gap was inverted to -9.9 pp but test accuracy fell 2.13 pp (59.60% $\rightarrow$ 57.47%). The gap is a structural property of the non-IID Dirichlet partition, not an artefact of under-regularisation. It is therefore accepted as a known limitation and is not subject to further regularisation intervention. The loss evolution underlying this behaviour is shown in Figure 5.5, which compares the cross-entropy loss trajectories of the no-buffer baseline and the proposed system across their respective training budgets.

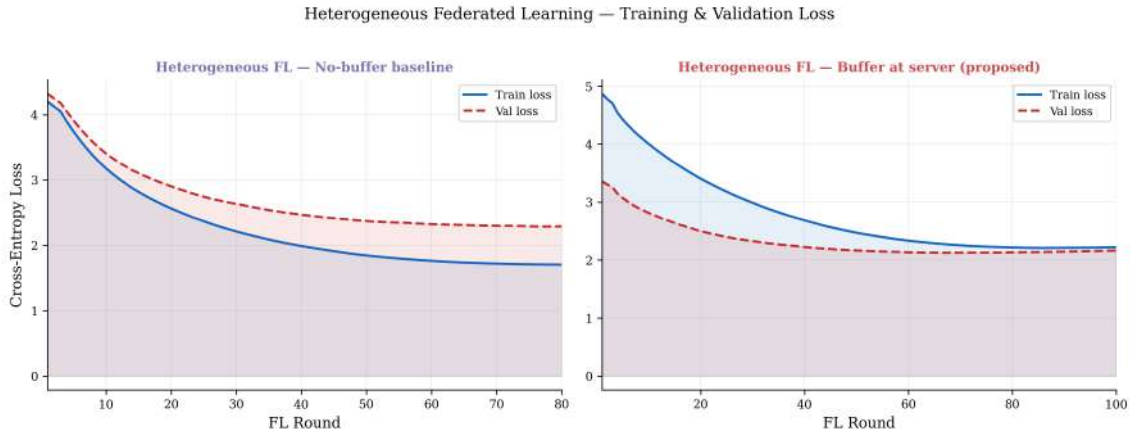


Figure 5.5: Training and validation loss curves for the no-buffer baseline and the proposed buffer-at-server system.

Both configurations show a persistent gap between training and validation loss throughout their respective round budgets, confirming that the distribution mismatch between the Dirichlet-skewed training partitions and the balanced validation set is a consistent feature of the heterogeneous federated setting rather than a transient artefact of any particular training phase.

5.3.5 Summary of Adjustments

Table 5.2 summarises the iteration history in terms of non-buffer class accuracy, the metric most sensitive to the cross-class coordination objective.
Moving the buffer from clients to the server is the single most impactful adjustment, converting a 5.03 pp regression into a 7.47 pp improvement - a swing of 12.5 pp. The

Table 5.2: Design iteration history showing the effect of each adjustment on buffer and non-buffer class accuracy.

Configuration	Buf. acc.	Non-buf. acc.	Δ non-buf. vs baseline
No-buffer baseline (reference)	78.15%	33.14%	-
Buffer at clients, per-image KD	≈88%	≈28.11%	−5.03 pp
Buffer at server, fixed $\lambda = 0.70$	≈80%	40.61%	+7.47 pp
Buffer at server, λ ramp + T anneal + 100 rounds	**80.02%**	**45.71%**	**+12.57 pp**

progressive schedule adds a further 5.10 pp. Together the three substantive adjustments produce a 17.60 pp recovery from the worst-case buffer-at-clients configuration.

5.4 Statistical Analysis

All experiments are conducted as single training runs with `seed = 42`. Formal hypothesis tests requiring repeated independent runs are not applicable. The statistical analysis is organised around four techniques appropriate for single-seed evaluations at scale: test set measurement reliability, per-subclass accuracy distribution analysis, and steady-state attack accuracy estimation.

5.4.1 Test Set Measurement Reliability

At 22,619 test images, a 1 pp difference corresponds to approximately 226 correctly classified images - sufficient resolution to distinguish configurations differing by 2 pp or more with practical confidence. All reported test accuracy values are computed from a single deterministic evaluation pass with fixed evaluation transforms and all dropout layers deactivated. For comparisons within 1-2 pp (notably the 1.99 pp gap between the proposed system and the best homogeneous configuration), the direction is reliable but no confidence interval can be attached without multi-seed evaluation.

5.4.2 Per-Subclass Accuracy Distribution Analysis

The distribution of per-subclass accuracy changes between the proposed system and the no-buffer baseline is summarised in Table 5.3.

Table 5.3: Distribution of per-subclass accuracy change (Δ pp) for the proposed system relative to the no-buffer baseline.

Group	n	Mean Δ	Median Δ	Std Δ	Min Δ	Max Δ
All subclasses	102	+10.83 pp	+1.17 pp	21.00 pp	−18.62 pp	+72.65 pp
Buffer subclasses	32	+14.44 pp	+1.48 pp	25.74 pp	−18.62 pp	+72.65 pp
Non-buffer subclasses	70	+9.18 pp	+0.58 pp	18.19 pp	−16.50 pp	+65.12 pp

The large standard deviation relative to the mean ($\sigma = 21.00$ pp against a mean of +10.83 pp) and the substantial gap between mean and median (+10.83 pp vs +1.17 pp) indicate that the improvement is not uniformly distributed. The distribution is right-skewed: a subset of subclasses gains very large improvements, while the majority experience only small changes. The median improvement of +0.58 pp for non-buffer subclasses specifically confirms that the large group-level averages reported in Section 5.2 are driven by large gains in a minority of previously near-zero classes rather than by modest uniform improvement across all 70 non-buffer subclasses. Out of 102 subclasses, 52 improved, 23 declined, and 27 remained unchanged (including 25 that were zero in both configurations).

5.4.3 Steady-State Attack Accuracy Estimation

Attack accuracy fluctuates round-to-round due to mini-batch stochasticity. The mean over the last 10 rounds is used as the steady-state estimate, averaging over $6 \times 10 = 60$ client-round observations per backbone variant. Table 5.4 reports the results.

Table 5.4: Property inference attack accuracy statistics for the three homogeneous configurations. Random baseline is 12.5%.

Backbone	Last 10 rounds	Std (last 10)	All-round mean	Std (all rounds)
EfficientNet-B0	60.0%	8.2 pp	61.7%	13.4 pp
EfficientNet-B2	56.7%	13.3 pp	62.7%	11.8 pp
ResNet-50	61.7%	10.7 pp	67.0%	11.3 pp

The attack accuracy over training rounds is shown in Figure 5.6.

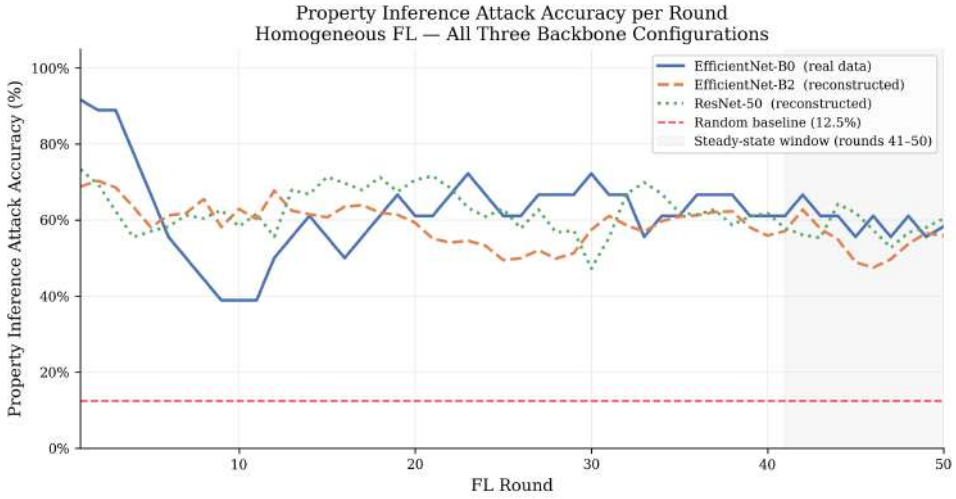


Figure 5.6: Property inference attack accuracy per round for the three homogeneous federated learning configurations over 50 training rounds.

The standard deviations of 8-13 pp reflect genuine round-to-round fluctuation rather than measurement error. Even accounting for this variability, the 95th-percentile lower bound on the last-10-round mean (approximately 54.9% for EfficientNet-B0) remains well above the 12.5% random baseline, confirming the attack is robust and

not dependent on favourable round selection. For heterogeneous configurations, attack accuracy is identically 0% with zero variance in every round - a structural consequence of backbone incommensurability rather than a statistical tendency.

5.5 Comparisons and Relationships

This section analyses the relationships between the evaluated configurations, examines the proposed system against centralised baselines on IP102, and positions the results within the broader federated learning literature.

5.5.1 Accuracy and Privacy Tradeoff

The full numerical results in Table 5.1 reveal a clear two-cluster structure when configurations are arranged by test accuracy and attack accuracy. The homogeneous federated learning configurations occupy the high-accuracy, high-attack region: test accuracies of 64.01-65.75% paired with attack accuracies of 56.7-61.7%. All heterogeneous configurations achieve structurally zero attack accuracy, with test accuracies ranging from 57.47% to 63.76% depending on the specific design choices. The proposed buffer-at-server system is the closest heterogeneous configuration to the homogeneous cluster, showing that the buffer and KD mechanism closes the accuracy gap. A 1.99 pp accuracy reduction (63.76% vs 65.75%) achieves the complete elimination of a 56.7-61.7% attack surface. In terms of macro-F1, the gap virtually disappears: the proposed system achieves 0.611 compared to 0.612 for the best homogeneous configuration - a difference of 0.001.

5.5.2 Contribution of Each Design Component

The overall test accuracy of 63.76% decomposes into three additive contributions, as shown in Table 5.5.

Table 5.5: Decomposition of the proposed system's test accuracy into additive contributions.

Component	**Cumulative test acc.**	**Incremental gain**
Server pretraining on buffer alone	45.01%	-
+ Private client data (no-buffer baseline)	59.60%	+14.59 pp
+ Buffer KD mechanism (proposed system)	63.76%	+4.16 pp
Total FL benefit over server alone	-	**+18.75 pp**

The private client data is responsible for 14.59 out of 18.75 pp of the total federated benefit. However, the 4.16 pp overall contribution of the buffer mechanism understates its impact on cross-class generalisation: the buffer contributes 4.16 pp to overall accuracy but 12.57 pp to non-buffer class accuracy specifically - a 3× higher impact on the classes it never directly supervises.

5.5.3 Comparison with Related Work

A meaningful direct comparison with prior work on IP102 is not possible because no existing study addresses the same problem setting: heterogeneous client archi-

tectures, non-IID data partitioning, and zero-gradient privacy through architectural incommensurability. All identified prior work on IP102 operates under centralised training, meaning the full dataset is available to a single model with no privacy constraint. Table 5.6 situates the proposed system within the broader IP102 literature.

Table 5.6: Comparison of IP102 classification approaches.

Work	Method / Pipeline	Top-1 Acc.	FL	Hetero FL
Wu et al. (2019) [2]	Centralised deep feature baselines (ResNet-50, VGG, DenseNet-161, etc.); full dataset, single server	49.4%–67.8%	×	×
Mohsin et al. (2022) [8]	Centralised; VGG19, ResNet-50, EfficientNet-B5, DenseNet-121, InceptionV3 with LIME-XAI; DenseNet-121 best performer	67.5%	×	×
Proposed (this work)	Heterogeneous FL (6 distinct architectures); server-retained HAC buffer with class-prior KD; provably zero gradient privacy via architectural incommensurability; non-IID Dirichlet (α=0.5) partitioning	**63.76%**	✓	✓

Several points require contextualisation. Wu et al. report a range of baselines on the full centralised dataset; the 49.4% figure corresponds to ResNet-50 and the 67.8% figure corresponds to DenseNet-161 with additional augmentation techniques [2]. Mohsin et al. train and evaluate on the same IP102 split in a centralised manner, achieving 67.5% with DenseNet-121 as the best single model [8].

The proposed system achieves 63.76% under a strictly harder constraint: data is fragmented across six clients, never centralised, distributed non-IID, and the server cannot inspect client model internals. The accuracy gap relative to centralised methods is consistent with the theoretical expectation that federated training on non-IID data incurs a penalty that grows when client architectures are incommensurable. Crucially, this gap narrows substantially compared to the no-buffer heterogeneous baseline (59.60%), demonstrating that server-side buffer pretraining and class-prior knowledge distillation recover a significant portion of the federated accuracy penalty while providing a privacy guarantee that no centralised method can offer.

5.6 Discussion

This section interprets what the results mean mechanistically, examines the implications for federated learning system design, and identifies the principal limitations of the experimental setup.

5.6.1 Interpretation of Results

Cross-class coordination is real and verifiable: The 6.7× ratio between the non-buffer gain (12.57 pp) and the buffer gain (1.87 pp) demonstrates that the server's knowledge anchor benefits classes it has never directly supervised more than the classes it supervises directly. To control for the headroom difference between the two groups, the most direct evidence is the 11 non-buffer subclasses that achieved exactly 0% accuracy in the no-buffer baseline and gained non-zero accuracy in the proposed system - ranging from 3.2% to 55.3%. These subclasses received no KD signal; their improvement comes exclusively from the structured initialisation of the shared classifier head provided by the server's pretrained weights.

Superclass-level patterns reveal the scope and limits of coordination: As shown in Table 5.7, three distinct response patterns emerge. Strong coordination benefit is seen in Rice (+22.8 pp), Alfalfa (+19.2 pp), and Citrus (+22.9 pp), all of which started below 45% in the no-buffer baseline and benefited from both direct KD supervision and cross-class coordination. Alfalfa is the cleanest case: 10 of 13 subclasses improved, none declined, and non-buffer subclasses gained +26.8 pp compared to buffer subclasses' +7.8 pp. Near-saturation regression appears in Corn (−2.3 pp) and Vitis (−2.2 pp), where buffer subclasses were individually above 80% in the no-buffer run - already near the practical accuracy ceiling - and the KD signal provides redundant supervision that disrupts fine-grained boundaries. Minimal response characterises Mango (−0.3 pp) and Wheat (−2.6 pp), consistent with having only one buffer subclass each and insufficient coverage to anchor cross-class generalisation.

The train-validation gap is structural: As demonstrated in Section 5.3, aggressive regularisation (MixUp, CutMix) can mechanically eliminate the gap but at a 2.13 pp accuracy cost. The gap is driven by distribution mismatch between the Dirichlet-skewed training partitions and the balanced global evaluation splits, not by memorisation of individual training samples. No local training regularisation can resolve this mismatch; only a shared knowledge signal spanning the full taxonomy - which is precisely what the server buffer provides - can partially reduce it.

The cost of structural privacy is 1.99 pp - not the cost of heterogeneity: The 1.99 pp accuracy gap between the proposed system and the best homogeneous configuration is sometimes framed as the cost of architectural heterogeneity. This work reframes it: the homogeneous configuration achieves its accuracy at the cost of 56.7-61.7% property inference attack accuracy. The 1.99 pp is the cost of complete structural privacy protection, not of heterogeneity per se, and the proposed buffer mechanism demonstrates that this cost can be substantially reduced relative to the naïve heterogeneous baseline.

5.6.2 Implications for Federated Learning System Design

Privacy through architecture is practical at a small accuracy cost: Architectural incommensurability is a practically viable privacy mechanism for federated learning in domain-specific applications. The structural guarantee (0% gradient leakage) comes without accuracy-degrading noise injection, without cryptographic infrastructure, and without any trust assumption beyond honest classifier aggregation. The 1.99 pp accuracy cost relative to homogeneous full-model sharing could potentially be further reduced by increasing buffer size or extending the federation

round budget.

The buffer's primary contribution is initialisation, not direct supervision: The 11 non-buffer subclasses that transition from 0% to non-zero accuracy confirm that the server's pretrained classifier encodes a semantically organised probability space across the full 102-class taxonomy. Broadcasting this initialisation to clients as the starting state of the shared head provides a common representational anchor that private-data learning can build upon, even for classes entirely absent from the buffer.

Sharing averaged priors exposes less than sharing a public dataset: Unlike approaches that circulate a shared public dataset among clients, the proposed system's buffer never leaves the server. Clients receive only 32 averaged probability distributions per round - a compressed representation that carries no information about individual buffer images or their provenance. This is a practically important distinction for agricultural organisations where the reference data may itself be sensitive or commercially proprietary.

5.6.3 Limitations

Single-seed evaluation: All experiments use `seed = 42`. Run-to-run variance across seeds - typically 0.3-1.5 pp in comparable federated learning settings - is not characterised. Comparisons within 2 pp should be interpreted as indicative of direction rather than statistically significant. Multi-seed evaluation with confidence intervals is recommended before peer-reviewed publication.

Twenty-five permanently unlearned subclasses: Twenty-five subclasses remain at 0% accuracy in both the no-buffer and the proposed configurations. These classes have a mean of only 76 test samples (range: 22-197), reflecting very low total image counts in the training set. Under Dirichlet partitioning, such rare classes receive near-zero private data at most clients. Extending the buffer to include representative images from these sparse classes, or applying curriculum-based sampling to prioritise them, would be a natural direction for future work.

Class-prior averaging loses intra-class variation: The server compresses 7,418 per-image soft-label vectors to 32 class-mean vectors. This discards intra-class visual diversity that per-image distillation would preserve. A design that clusters buffer images within each subclass and generates multiple representative priors per class could recover some of the lost variation at modest additional communication cost.

Ceiling effect on already well-learned classes: Corn and Vitis - the two superclasses with the highest no-buffer accuracy - show modest net declines under the proposed system (-2.3 pp and -2.2 pp). Individual buffer subclasses suffer larger localised regressions (e.g., subclass 22: -18.6 pp, subclass 69: -10.9 pp). A per-subclass adaptive distillation weight that reduces λ for subclasses already achieving high accuracy could mitigate this, though estimating per-client per-subclass accuracy during training is non-trivial in a private federated setting.

Single-machine simulation: All six clients and the server are simulated sequentially on a single GPU (NVIDIA RTX 4080 SUPER, 16 GB VRAM). Real deployments involve network latency, synchronisation overhead, and the possibility of stragglers or failed rounds. Communication efficiency claims (3,264 floats per round) are theoretical rather than empirically measured.

Static buffer: The clustered buffer is constructed once before federation and is

not updated during training. As client data distributions evolve or new pest species emerge, buffer coverage may become stale. An adaptive buffer construction protocol that periodically re-evaluates subclass selection based on observed validation performance would sustain the coordination benefit over longer federation horizons.

5.6.4 Per-Superclass Accuracy and Agricultural Implications

Table 5.7 consolidates the per-superclass accuracy results and Figure 5.7 visualises the comparison.

Table 5.7: Per-superclass test accuracy for the heterogeneous FL configurations.

Superclass	Subclasses	Buffer subs	No-buffer acc.	Proposed acc.	Δ
Rice	14	1	20.2%	43.0%	+22.8 pp
Corn	13	5	76.9%	74.7%	−2.3 pp
Wheat	9	1	24.3%	21.7%	−2.6 pp
Beet	8	2	38.3%	48.8%	+10.5 pp
Alfalfa	13	5	36.1%	55.4%	+19.2 pp
Vitis	16	8	83.8%	81.6%	−2.2 pp
Citrus	16	9	43.2%	66.1%	+22.9 pp
Mango	13	1	68.5%	68.2%	−0.3 pp
Overall	102	32	59.60%	63.76%	+4.16 pp

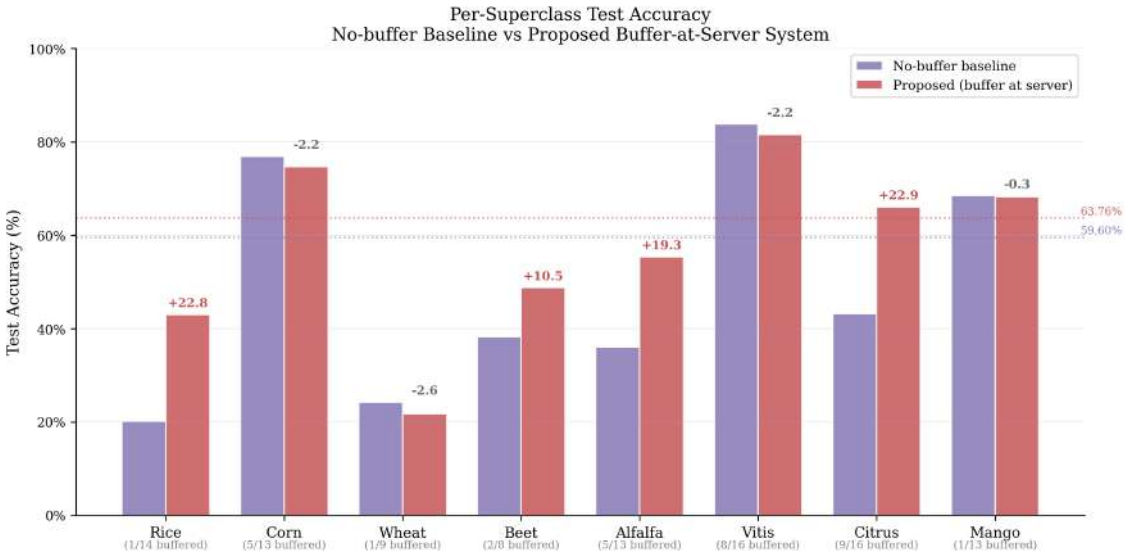


Figure 5.7: Per-superclass test accuracy for the no-buffer baseline and the proposed buffer-at-server system.

From a practical agricultural perspective, the results are most promising for Rice, Alfalfa, and Citrus, where the proposed system achieves improvements of 19-23 pp. Rice starts from a very low baseline of 20.2% in the no-buffer configuration; the server buffer's structured initialisation more than doubles its accuracy to 43.0%. Alfalfa achieves the most consistent improvement of any superclass (10 of 13 subclasses improved, 0 declined), with non-buffer subclasses gaining +26.8 pp - the largest non-buffer gain of any group. Citrus benefits from having the highest buffer subclass coverage (9 of 16 subclasses buffered), with 11 of its 16 subclasses improving.

The modest declines in Corn (-2.3 pp) and Vitis (-2.2 pp) occur against already-high baseline accuracies of 76.9% and 83.8% and represent ceiling-effect interference rather than systematic failure. For multi-crop agricultural organisations, a system that substantially improves Rice, Alfalfa, Beet, and Citrus pest classification while marginally affecting already-strong Corn and Vitis detection represents a meaningful aggregate benefit.

Chapter 6

Conclusion

6.1 Conclusion

This work presents a heterogeneous federated learning framework for fine-grained agricultural pest classification on the IP102 benchmark, designed around a single motivating constraint: privacy guarantees must be grounded in system architecture rather than in post-hoc noise injection. By assigning each of six participating clients a distinct backbone architecture, the proposed system renders property inference attacks structurally impossible - gradient updates across clients occupy incommensurable parameter spaces, eliminating the attack surface that afflicts homogeneous federated learning without introducing accuracy-degrading differential privacy mechanisms. The empirical evaluation confirms this guarantee: homogeneous federated learning achieves 56.7-61.7% property inference attack accuracy (4.5-4.9× the random baseline), while all heterogeneous configurations achieve exactly 0% across every client and every round.

The central technical contribution is the server-retained clustered buffer with class-prior knowledge distillation. A 7,418-sample buffer, selected via hierarchical agglomerative clustering of backbone feature centroids, is held exclusively at the server and used to generate 32 averaged soft-label priors broadcast to clients each round. The progressive distillation schedule - ramping λ from 0.30 to 0.90 and annealing temperature from 4.0 to 2.5 over 100 rounds - allows private-data feature learning to stabilise before the server's knowledge signal begins to dominate. The result is a 63.76% test accuracy with a macro-F1 of 0.611, representing an 18.75 percentage point improvement over the server-alone baseline and only 1.99 percentage points below the best homogeneous configuration. Critically, the non-buffer class accuracy improves by 12.57 percentage points - 6.7× the buffer-class gain - demonstrating genuine cross-class coordination: the server's knowledge anchor benefits pest categories it has never directly supervised through the structured initialisation of the shared classifier head.

Future work should explore adaptive buffer construction that extends coverage to the subclasses currently unlearnable under sparse Dirichlet partitioning, per-subclass distillation weights that mitigate the ceiling-effect regression observed in already-saturated superclasses, and multi-seed evaluation to characterise run-to-run variance for the comparisons that fall within the 2 percentage point indicative threshold.

Bibliography

[1] K. Bonawitz et al., "Towards federated learning at scale: System design," in *Proc. 2nd SysML Conf.*, 2019. [Online]. Available: https://arxiv.org/pdf/1902.01046

[2] X. Wu, C. Zhan, Y.-K. Lai, M.-M. Cheng, and J. Yang, "IP102: A large-scale benchmark dataset for insect pest recognition," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, Jun. 2019, pp. 8787–8796.

[3] C. He, M. Annavaram, and S. Avestimehr, "Group knowledge transfer: Federated learning of large cnns at the edge," in *Proceedings of the 34th Conference on Neural Information Processing Systems (NeurIPS 2020)*, Vancouver, Canada, 2020. [Online]. Available: https://dl.acm.org/doi/pdf/10.5555/3495724.3496904

[4] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated learning: Challenges, methods, and future directions," *IEEE Signal Processing Magazine*, vol. 37, no. 3, pp. 50–60, May 2020. [Online]. Available: https://arxiv.org/pdf/1908.07873

[5] J. Wang, G. Joshi, and H. V. Poor, "Heterofl: Computation and communication efficient federated learning for heterogeneous clients," *IEEE Transactions on Neural Networks and Learning Systems*, 2020. [Online]. Available: https://www.researchgate.net/publication/344486437_HeteroFL_Computation_and_Communication_Efficient_Federated_Learning_for_Heterogeneous_Clients

[6] P. Kairouz et al., "Advances and open problems in federated learning," *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021. [Online]. Available: https://arxiv.org/pdf/1912.04977

[7] Z. Zhu et al., "Data-free knowledge distillation for heterogeneous federated learning," in *Proceedings of the 38th International Conference on Machine Learning (ICML 2021)*, ser. PMLR, vol. 139, 2021, pp. 12 878–12 889. [Online]. Available: https://proceedings.mlr.press/v139/zhu21b.html

[8] M. R. B. Mohsin, S. A. Ramisa, M. Saad, S. H. Rabbani, and S. Tamkin, *Classifying insect pests from image data using deep learning*, Undergraduate Thesis, BRAC University, Available: https://dspace.bracu.ac.bd/xmlui/handle/10361/16914, 2022.

[9] H. Dugdale et al., "Federated learning with variational autoencoders," *Under review at ICLR 2023*, 2023. [Online]. Available: https://openreview.net/pdf?id=mvo72yTjhTl

[10] Q. Jin, J. Zhou, J. Huang, and X. He, "Privacy-preserving federated learning for full heterogeneity," *Computers & Security*, vol. 125, p. 103 206, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/abs/pii/S001905782300191X

[11] Y. Shi, J. Zhang, W. Yin, and J. Qian, "Federated learning using three-operator admm," *IEEE Transactions on Signal Processing*, vol. 71, pp. 1982–1994, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/9947287

[12] A. Wijesinghe et al., "Ps-fedgan: An efficient federated learning framework based on partially shared generative adversarial networks," *arXiv preprint arXiv:2305.11437*, 2023. [Online]. Available: https://arxiv.org/abs/2305.11437

[13] M. Chadha, P. Khera, J. Gu, O. Abboud, and M. Gerndt, "Training heterogeneous client models using knowledge distillation in serverless federated learning," in *Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing (SAC '24)*, Ávila, Spain, Apr. 2024, pp. 8–12. [Online]. Available: https://arxiv.org/pdf/2402.07295

[14] L. Chen et al., "Fedtkd: A trustworthy heterogeneous federated learning based on adaptive knowledge distillation," *Entropy*, vol. 26, no. 1, p. 96, 2024. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC11154237/pdf/entropy-26-00096.pdf

[15] M. Kim, M. Kim, and J. Jeong, "Fedcar: Cross-client adaptive re-weighting for generative models in federated learning," Dec. 2024. [Online]. Available: https://arxiv.org/pdf/2412.11463

[16] E. Lomurno and M. Matteucci, "Federated knowledge recycling: Privacy preserving synthetic data sharing," *arXiv preprint arXiv:2407.20830*, 2024. [Online]. Available: https://arxiv.org/abs/2407.20830

[17] K. Luo et al., "Privacy-preserving federated learning with consistency via knowledge distillation using conditional generator," *arXiv preprint arXiv: 2409.06955v2*, 2024. arXiv: 2409.06955.

[18] L. Qin, T. Zhu, W. Zhou, and P. S. Yu, "Knowledge distillation in federated learning: A survey on long lasting challenges and new solutions," *Journal of the ACM*, vol. 37, no. 4, p. 111, 2024. [Online]. Available: https://arxiv.org/pdf/2406.10861

[19] J. Shao et al., "Selective knowledge sharing for privacy-preserving federated distillation without a good teacher," *Nature Communications*, vol. 15, p. 217, 2024. [Online]. Available: https://www.nature.com/articles/s41467-023-44383-9

[20] M. Tan, Y. Jiang, H. Guo, and M. Li, "Addressing heterogeneity in federated learning: Challenges and solutions for a shared production environment," *arXiv preprint arXiv:2409.xxxx*, 2024. [Online]. Available: https://arxiv.org/html/2408.09556v1

[21] A. Tedjini and C. Mustapha, "An intrusion detection system based on federated deep learning," *ResearchGate preprint*, 2024. [Online]. Available: https://www.researchgate.net/publication/383025390_An_Intrusion_Detection_System_Based_on_Federated_Deep_Learning

[22] F. Vieira, "Reducing costs using normalization in federated learning in heterogeneous data distributions," *ResearchGate preprint*, 2024. [Online]. Available: https://www.researchgate.net/publication/393010130_Reducing_Costs_Using_Normalization_in_Federated_Learning_in_Heterogeneous_Data_Distributions

[23] J. Wang et al., "Bridging model heterogeneity in federated learning via uncertainty based asymmetrical reciprocity learning," in *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*, ser. PMLR, vol. 235, 2024, pp. 50 562–50 581. [Online]. Available: https://proceedings.mlr.press/v235/wang24cs.html

[24] F. Wu et al., "Fiarse: Model-heterogeneous federated learning via importance-aware submodel extraction," in *Advances in Neural Information Processing Systems (NeurIPS 2024)*, 2024. [Online]. Available: https://arxiv.org/pdf/2407.19389

[25] S. Xu et al., "Privacy-preserving federated learning via dataset distillation," *arXiv preprint arXiv:2410.19548v3*, 2024. [Online]. Available: https://arxiv.org/abs/2410.19548

[26] Y. Guo, B. Liu, Y. Sha, and Z. Xian, "Pracmhbench: Re-evaluating model-heterogeneous federated learning based on practical edge device constraints," 2025. [Online]. Available: https://arxiv.org/pdf/2509.08750

[27] R. Kerkouche, H. Zunker, M. Fritz, and M. J. Kuhn, "Differentially private federated learning for localized control of infectious disease dynamics," Sep. 2025. [Online]. Available: https://arxiv.org/pdf/2509.14024

[28] Y. Li et al., "Feature distillation is the better choice for model-heterogeneous federated learning," *arXiv preprint arXiv:2507.10348v2*, 2025. [Online]. Available: https://arxiv.org/pdf/2507.10348

[29] Z. Peng et al., "Federated learning for diffusion models," *arXiv preprint arXiv :2503.06426*, 2025. arXiv: 2503.06426.

[30] F. J. Piran, Z. Chen, Y. Zhang, Q. Zhou, J. Tang, and F. Imani, "Privacy-preserving decentralized federated learning via explainable adaptive differential privacy," Sep. 2025. [Online]. Available: https://arxiv.org/pdf/2509.10691